

Apostila Final — Teoria da Informação

Contents

Apostila Final — Teoria da Informação	4
Sumário	4
PARTE I — FUNDAMENTOS	5
Capítulo 1 — Dados, Informação e Conhecimento	6
1.1 Por que começar por aqui?	6
1.2 A cadeia Dado $\rightarrow$ Valor $\rightarrow$ Informação $\rightarrow$ Conhecimento	6
1.3 A Era da Informação: de “poucos dados” a “excesso de dados”	6
1.4 Tipos de dados: estruturados, semi-estruturados e não estruturados	7
1.5 Vocabulário: atributos e objetos	8
1.6 O pipeline completo: da coleta ao conhecimento	9
Capítulo 2 — Fundamentos de Probabilidade	10
2.1 Axiomas de probabilidade (Kolmogorov)	10
2.2 Probabilidade condicional e independência	10
2.3 Função de distribuição, densidade e variáveis discretas/contínuas	11
2.4 Valor esperado, momentos, variância	11
2.5 Covariância e correlação	12
2.6 Mudança de variáveis (Jacobiano)	12
2.7 Três desigualdades fundamentais	12
Capítulo 3 — Estatística Descritiva e Pré-processamento de Dados	14
3.1 Medidas de posição	14
3.2 Medidas de dispersão	14
3.3 Distribuição e frequência	15
3.4 Quartis e percentis	15
3.5 Correlação amostral	15
3.6 Recomendações de uso de métricas	15
3.7 Transformação de dados	16
3.8 Normalização de dados	16
Capítulo 4 — Distribuições de Probabilidade Contínuas	18
4.1 Distribuição Exponencial e a propriedade de falta de memória	18
4.2 Distribuição Gama e sua relação com Exponencial, Erlang e Qui-quadrado	19
4.3 Relação Gama–Beta	19
4.4 Weibull como distribuição “camaleão”	19

4.5 Distribuição Normal e o Teorema do Limite Central	19
Capítulo 5 — Inferência, Processos Estocásticos e Técnicas de Predição	20
5.1 Teorema de Bayes	20
5.2 Processos estocásticos e Cadeias de Markov	20
5.3 Teoria da decisão	21
5.4 Técnicas de predição (visão panorâmica)	21
PARTE II — TEORIA DA INFORMAÇÃO CLÁSSICA (SHANNON)	23
Capítulo 6 — Quantidade de Informação e Entropia	24
6.1 Autoinformação (informação de um evento)	24
6.2 Entropia de Shannon	25
6.3 Extensão de uma fonte discreta sem memória	25
Capítulo 7 — Entropia Conjunta, Condicional, Divergência KL e Informação Mútua	26
7.1 Entropia conjunta	26
7.2 Entropia condicional	26
7.3 Regra da cadeia	26
7.4 Divergência de Kullback-Leibler (entropia relativa / entropia cruzada)	27
7.5 Informação Mútua	27
7.6 Ganho de Informação e árvores de decisão	27
Capítulo 8 — Entropia Diferencial e Generalizações da Entropia	29
8.1 Entropia diferencial	29
8.2 Propriedades gerais (válidas nos dois casos)	29
8.3 Outras definições de entropia	30
Capítulo 9 — Codificação de Fonte e Compressão de Dados	32
9.1 O problema da codificação de fonte	32
9.2 O Primeiro Teorema de Shannon (Teorema da Codificação de Fonte)	32
9.3 Códigos prefixo e a Inequação de Kraft-McMillan	32
9.4 Codificação de Huffman	33
9.5 Codificação Lempel-Ziv-Welch (LZW)	34
9.6 Compressão com e sem perdas — visão geral	35
9.7 Estimando entropia a partir de dados reais	35
Capítulo 10 — Capacidade de Canal, Ruído e o Limite de Shannon	36
10.1 O problema da codificação de canal	36
10.2 Segundo Teorema de Shannon (Teorema da Codificação do Canal)	36
10.3 Capacidade de canal: definição geral	37
10.4 Terceiro Teorema de Shannon: capacidade do canal gaussiano (Shannon-Hartley)	37
Capítulo 11 — Códigos Corretores de Erro	40
11.1 Motivação	40
11.2 Categorias de códigos	40
11.3 Técnicas de detecção e correção	40
11.4 Taxa de código e o trade-off fundamental	41
11.5 Aplicações modernas	41

Capítulo 12 — Complexidade de Kolmogorov	42
12.1 Uma segunda forma de medir informação	42
12.2 Definição formal	42
12.3 Strings incompressíveis	43
12.4 O Teorema de Gödel via Complexidade de Kolmogorov	43
12.5 Aleatoriedade formalizada sem probabilidade	44
12.6 Complexidade de Kolmogorov como precursor da entropia	44
 PARTE III — BIG DATA, APRENDIZADO DE MÁQUINA E RECONHECI- MENTO DE PADRÕES	 45
 Capítulo 13 — Big Data: Frameworks e Processamento Distribuído	 46
13.1 Por que processamento distribuído?	46
13.2 MapReduce	46
13.3 Processamento em fluxo (Streaming) e Apache Spark	47
13.4 Convergência com IA	47
 Capítulo 14 — Fundamentos de Redes Neurais e Aprendizado	 49
14.1 Por que aprendizado de máquina?	49
14.2 Métricas de avaliação	49
14.3 O mecanismo de treinamento	50
14.4 Overfitting e boas práticas	51
 Capítulo 15 — Análise de Componentes Principais (PCA)	 52
15.1 Contexto histórico	52
15.2 O problema: redução de dimensionalidade preservando informação	52
15.3 O algoritmo PCA, passo a passo	52
15.4 Por que isso funciona? A diagonalização da covariância	53
15.5 Interpretação geométrica: mudança de base	53
15.6 Aplicações	54
 Capítulo 16 — Síntese Final e Mapa Conceitual	 55
16.1 O fio condutor do curso	55
16.2 Mapa de conexões entre os capítulos	55
16.3 Tabela-resumo das grandezas fundamentais	56
16.4 Dicas de estudo e revisão	56
Referências e leituras complementares	57

Apostila Final — Teoria da Informação

Material consolidado a partir das palestras, slides e apostilas da disciplina

Esta apostila reúne, organiza e complementa didaticamente todo o conteúdo apresentado ao longo do curso: os slides de aula do Prof. Dr. Julio César Santos dos Anjos (módulos 01 a 06 de “Tecnologia da Informação”), a apostila de estatística da UFC, o material de PCA e o capítulo de Complexidade de Kolmogorov (Durand & Zvonkin). O objetivo é oferecer um texto único, sequencial e autocontido — útil tanto para quem está tendo o primeiro contato com a disciplina quanto para quem já cursou e está revisando parcial ou totalmente o conteúdo.

Como usar: os capítulos seguem uma progressão lógica (dos fundamentos de dados/probabilidade até as aplicações em Big Data e Machine Learning), mas cada capítulo é relativamente autocontido, podendo ser consultado isoladamente para revisão. Fórmulas estão em LaTeX; a maioria dos ambientes de leitura Markdown modernos (Obsidian, Typora, VS Code com extensão de matemática, GitHub, etc.) renderiza esse formato automaticamente.

Sumário

Parte I — Fundamentos 1. Dados, Informação e Conhecimento 2. Fundamentos de Probabilidade 3. Estatística Descritiva e Pré-processamento de Dados 4. Distribuições de Probabilidade Contínuas 5. Inferência, Processos Estocásticos e Técnicas de Predição

Parte II — Teoria da Informação Clássica (Shannon) 6. Quantidade de Informação e Entropia 7. Entropia Conjunta, Condicional, Divergência KL e Informação Mútua 8. Entropia Diferencial e Generalizações da Entropia 9. Codificação de Fonte e Compressão de Dados 10. Capacidade de Canal, Ruído e o Limite de Shannon 11. Códigos Corretores de Erro 12. Complexidade de Kolmogorov

Parte III — Big Data, Aprendizado de Máquina e Reconhecimento de Padrões 13. Big Data: Frameworks e Processamento Distribuído 14. Fundamentos de Redes Neurais e Aprendizado 15. Análise de Componentes Principais (PCA)

Fechamento 16. Síntese Final e Mapa Conceitual

PARTE I — FUNDAMENTOS

Capítulo 1 — Dados, Informação e Conhecimento

1.1 Por que começar por aqui?

Antes de qualquer formalismo matemático, a disciplina começa com uma pergunta aparentemente simples: **o que é informação?** Essa pergunta tem uma longa história — desde os primeiros registros humanos (como as pinturas rupestres do Parque da Pedra Pintada, na Amazônia) até os bilhões de gigabytes gerados por segundo hoje em dia, a humanidade sempre lidou com o problema de **registrar, transmitir e interpretar** dados.

A ideia central que percorre todo o curso é que **dado bruto não é a mesma coisa que informação útil**. Um número, uma imagem ou um sinal, isolados, não “significam” nada por si só — é o processamento, o contexto e a interpretação que transformam dado em informação, e informação em conhecimento.

1.2 A cadeia Dado → Valor → Informação → Conhecimento

Essa progressão (uma variação da conhecida hierarquia **DIKW** — *Data, Information, Knowledge, Wisdom* — da Ciência da Informação) pode ser resumida assim:

- **Dado:** um registro bruto que, por si só, não carrega significado. Exemplo: o número 37.
- **Valor:** propriedade que se agrega ao dado para lhe dar relevância dentro de um contexto. Exemplo: 37 é uma **temperatura**.
- **Informação:** é o dado ao qual se atribuiu significado — um valor interpretado dentro de um contexto. Exemplo: “a temperatura corporal do paciente às 8h era 37 °C”.
- **Conhecimento:** resultado do cruzamento de diferentes informações, do qual se pode inferir e tomar decisões, gerando por sua vez **nova informação** (um ciclo de retroalimentação). Exemplo: cruzando a temperatura de 37 °C com o histórico do paciente e outros sintomas, um médico conclui que há um quadro febril e decide um tratamento — essa decisão gera novos dados/informações que realimentam o ciclo.

Esse ciclo (dado → conhecimento → nova informação) não é linear, é **iterativo**: o conhecimento gerado hoje se torna insumo (dado) para análises futuras.

1.3 A Era da Informação: de “poucos dados” a “excesso de dados”

Historicamente, a preocupação central da humanidade em relação à informação mudou de natureza:

Época	Problema central	Marco tecnológico
Passado remoto	Falta de informação — como transmiti-la?	Tradição oral
Invenção da escrita	Como representá-la de forma durável?	Escrita, depois imprensa
Era digital / Big Data	Excesso de informação — como processá-la?	Computadores, internet, sensores

Hoje vivemos o polo oposto do problema original: não falta dado, sobra. Alguns números que ilustram essa explosão (citados no material do curso, com fonte [statista.com](https://www.statista.com) e [discovery.com](https://www.discovery.com)):

- O volume mundial de dados gerados é medido em **zettabytes** (1 ZB = 10^9 TB).
- Há **6,65 bilhões** de usuários de smartphone no mundo — mais de 83% da população — e **60%** do tráfego de internet vem de dispositivos móveis.
- Cada olho humano capta cerca de **576 megapixels** por cena — quase 9 vezes mais que os 63 MP de uma câmera de iPhone 15 Pro. Essa comparação é usada para argumentar que **lidar com grandes volumes de dados é, em certo sentido, “natural”**: o próprio cérebro humano constantemente filtra e comprime informação sensorial bruta em percepções úteis.

A metáfora do iceberg: os dados que efetivamente coletamos e vemos são apenas a ponta visível de um iceberg — a maior parte do “valor” informacional está latente e só emerge depois de processamento, análise e modelagem. Essa é a motivação central para todo o restante do curso: aprender técnicas (estatísticas, de teoria da informação e de aprendizado de máquina) capazes de extrair esse valor oculto.

1.4 Tipos de dados: estruturados, semi-estruturados e não estruturados

Uma classificação central em Ciência de Dados, usada ao longo de todo o curso:

(a) Dados estruturados - Organização rígida e bem definida, tipicamente em **tabelas** (linhas e colunas). - Fácil de consultar e processar, mas caro de reestruturar. - Exemplos: bancos de dados relacionais (SQL), planilhas.

(b) Dados semi-estruturados - Não seguem um modelo tabular fixo, mas têm **metadados** (tags, marcadores) que permitem alguma organização. - Exemplos clássicos: XML e JSON. Compare a mesma informação nos dois formatos:

```
<user>
  <id>11</id>
  <animal>Bob</animal>
  <idade>2</idade>
</user>

{
  "user": { "id": "11", "animal": "Bob", "idade": "2" }
}
```

(c) **Dados não estruturados** - Sem padrão definido, não organizáveis em tabelas diretamente. - Cerca de **80% dos dados produzidos no mundo** são não estruturados (estatística amplamente citada na literatura de Big Data). - Exemplos: e-mails, textos livres, páginas web, imagens, áudio, vídeo — normalmente exigem uma etapa de **pré-processamento** antes de qualquer análise.

1.5 Vocabulário: atributos e objetos

Um **atributo** é uma propriedade ou característica de um objeto (também chamado de **variável**, **campo**, **característica** ou **feature** — termos usados de forma intercambiável em diferentes comunidades: estatística, bancos de dados, aprendizado de máquina). Exemplos: cor dos olhos, temperatura.

Um **objeto** é uma coleção de atributos (também chamado de **registro**, **ponto**, **caso**, **amostra**, **entidade** ou **instância**). Quando todos os objetos compartilham o mesmo conjunto de atributos numéricos, podem ser vistos como **pontos em um espaço multidimensional** — cada atributo é uma dimensão — e organizados em uma **matriz** $m \times n$ (m objetos/linhas, n atributos/colunas). Essa é exatamente a representação usada mais adiante em PCA (Capítulo 15) e em quase todo algoritmo de aprendizado de máquina.

Classificação dos atributos

Categoria	Subtipo	Característica	Exemplos
Categórico	Nominal	Sem ordem	Cor dos olhos, forma de pagamento
	Ordinal	Ordenável, mas não comparável numericamente	Ranking (bom/médio/ruim), faixa etária
Numérico	Discreto	Contagem	Número de filhos, idade em anos
	Contínuo	Escala contínua	Temperatura, peso
	Binário	Caso especial	Sim/não, existe/não existe

Essa taxonomia (equivalente às clássicas **escalas de medição de Stevens**: nominal, ordinal, intervalar, razão) determina, mais adiante, quais métricas estatísticas e quais técnicas de normalização fazem sentido para cada variável (ver Capítulo 3).

Formas de representação de dados

- **Documentos** (bag-of-words): cada documento vira um vetor cujo i -ésimo componente é o número de ocorrências do i -ésimo termo do vocabulário — perde a ordem das palavras, mas é útil para muitas tarefas de PLN (Processamento de Linguagem Natural).
- **Transações**: cada registro é um conjunto de itens, representável como vetor binário de presença/ausência — base da análise de cesta de compras (*market basket analysis*).
- **Dados ordenados**: sequências (ex.: genoma, séries temporais).
- **Grafos**: nós e arestas (ex.: links entre páginas web, redes sociais, redes de citação).

1.6 O pipeline completo: da coleta ao conhecimento

O curso apresenta um pipeline de cinco etapas para transformar dado bruto em conhecimento (uma versão prática do processo clássico de **KDD** — *Knowledge Discovery in Databases* — embora os slides originais não usem esse nome):

1. **Seleção**: extração dos dados de sua origem (pode ser um processo contínuo/streaming em sistemas de Big Data, e frequentemente recursivo, coletando em diferentes níveis/fontes).
2. **Pré-processamento**: limpeza (remoção de ruído/inconsistências), integração (combinação de múltiplas fontes) e seleção/redução dos dados relevantes. Os problemas típicos encontrados aqui são:
 - **Incompletude**: falta de valores, atributos ou objetos inteiros.
 - **Inconsistência**: valores fora do domínio esperado ou muito discrepantes.
 - **Ruído**: variações indesejadas e nem sempre identificáveis (ex.: “chuvisco” em vídeo, “chiado” em áudio).
3. **Transformação**: converter dados descritivos em formato quantitativo e padronizado — por exemplo, transformar um horário “8h–12h, 14h–17h” em categorias “manhã/tarde” (ou em códigos “0”/“1”), ou extrair *features* de uma imagem via convolução. Inclui a **normalização**, tratada em detalhe no Capítulo 3.
4. **Extração de conhecimento (mineração)**: aplicação de algoritmos de análise **descritiva** (tendência central, dispersão, visualização), **agrupamento** (clustering), **predição** (classificação/regressão), **associação** (quais atributos ocorrem juntos) e **detecção de anomalias**.
5. **Interpretação**: avaliação dos resultados, buscando conhecimento verdadeiramente útil e não trivial. Se os resultados não forem satisfatórios, **volta-se às etapas 1 ou 2** — o pipeline é **cíclico**, não estritamente sequencial, especialmente em aprendizado supervisionado/semi-supervisionado.

Esse ciclo de retroalimentação fecha a narrativa conceitual do capítulo: dado bruto → (pipeline) → conhecimento → nova informação, realimentando o processo.

Motivação real (exemplo astronômico): em 2009, o telescópio Hubble ajudou a identificar o exoplaneta **GJ1214b** (a 40 anos-luz da Terra); somente após um longo processo de análise dos dados brutos de trânsito planetário — dois anos depois — pesquisadores confirmaram, em 2011, que o planeta continha mais água que a Terra. O exemplo ilustra concretamente a tese central deste capítulo: **dado bruto** $\neq$ **informação útil**; é o processamento que revela o conhecimento.

Capítulo 2 — Fundamentos de Probabilidade

Toda a Teoria da Informação se apoia em probabilidade: entropia é definida a partir de probabilidades de símbolos, capacidade de canal a partir de distribuições de entrada/saída, etc. Este capítulo revisa o ferramental necessário.

2.1 Axiomas de probabilidade (Kolmogorov)

Seja S um espaço amostral (conjunto de todos os resultados possíveis) contido em um espaço de eventos F (uma coleção — tecnicamente uma σ -álgebra — de todos os subconjuntos de S aos quais se pode atribuir uma probabilidade). Para eventos mutuamente exclusivos $A \cdot B = \emptyset$ (ou seja, $A \cap B = \emptyset$):

$$P(A) \geq 0, \quad P(S) = 1, \quad A \cdot B = \emptyset \implies P(A + B) = P(A) + P(B)$$

Esses são os três **axiomas de Kolmogorov** (1933): não-negatividade, normalização (a probabilidade do espaço todo é 1) e aditividade para eventos disjuntos. Toda a teoria de probabilidade moderna é construída a partir desses três postulados.

2.2 Probabilidade condicional e independência

Probabilidade condicional — probabilidade de A dado que B já ocorreu (o espaço amostral é “restrito” a B):

$$P(A|B) \equiv \frac{P(AB)}{P(B)}, \quad P(B) > 0$$

Eventos independentes: A e B são independentes se conhecer a ocorrência de um não altera a probabilidade do outro:

$$P(A \cdot B) = P(A) \cdot P(B)$$

Generalizando para três eventos, independência **mútua** exige tanto independência **par a par** ($P(AB) = P(A)P(B)$, etc.) quanto $P(ABC) = P(A)P(B)P(C)$.

i.i.d. (independentes e identicamente distribuídos): termo central em Teoria da Informação (fontes emitem símbolos i.i.d.). São, na verdade, **duas** propriedades combinadas: *independência* (a probabilidade conjunta é o produto das marginais) e *distribuição idêntica* (todas as variáveis da sequência seguem a mesma distribuição de probabilidade).

2.3 Função de distribuição, densidade e variáveis discretas/contínuas

Uma variável aleatória pode ser: - **Discreta:** assume um número contável de valores (ex.: número de caras em 10 lançamentos). - **Contínua:** assume valores em um conjunto incontável (ex.: um tempo medido em segundos, com precisão arbitrária).

Função de distribuição acumulada (CDF):

$$F(x) = P(X \leq x)$$

com as propriedades: $\lim_{x \rightarrow -\infty} F(x) = 0$, $\lim_{x \rightarrow \infty} F(x) = 1$, F é não-decrescente e contínua à direita.

Para variáveis contínuas, define-se a **função densidade de probabilidade (PDF):**

$$f(x) = \frac{dF(x)}{dx}, \quad F(x) = \int_{-\infty}^x f(u) du, \quad P(a \leq X \leq b) = \int_a^b f(x) dx$$

Ponto conceitual importante (e contraintuitivo para iniciantes): para variáveis contínuas, $P(X = a) = 0$ para qualquer valor específico a — mesmo que a densidade $f(a)$ não seja zero. A densidade descreve *concentração* de probabilidade, não probabilidade em si. Exemplo: a chance de alguém pesar *exatamente* 70,000000... kg é zero, mas a chance de pesar entre 69 e 71 kg é positiva.

2.4 Valor esperado, momentos, variância

Valor esperado (média):

$$E[X] = \sum_i x_i f(x_i) \quad (\text{discreto}), \quad E[X] = \int_{-\infty}^{\infty} x f(x) dx \quad (\text{contínuo})$$

O valor $E[X^r]$ é chamado de **r -ésimo momento** de X (momento em relação à origem). Os **momentos centrais** são tomados em relação à média: $E[(X - \mu)^r]$. A **variância** é o momento central de segunda ordem:

$$Var(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2, \quad \sigma = \sqrt{Var(X)} \quad (\text{desvio padrão})$$

Propriedades da variância: 1. $Var(cX) = c^2 Var(X)$ — escalar a variável multiplica a variância pelo quadrado da escala (ex.: mudar de metros para centímetros multiplica Var por 10 000). 2. $Var(X + k) = Var(X)$ — deslocar (somar constante) não afeta a dispersão. 3. $Var(X) \geq 0$, com igualdade se e somente se X é degenerada (constante).

2.5 Covariância e correlação

$$\text{Cov}(X, Y) = E[(X - \mu_x)(Y - \mu_y)] = E[XY] - E[X]E[Y]$$

Se X, Y são independentes, $\text{Cov}(X, Y) = 0$ — mas **a recíproca não vale em geral**: covariância nula não implica independência (apenas ausência de dependência *linear*).

Correlação de Pearson (normaliza a covariância pelos desvios-padrão, ficando sempre em $[-1, 1]$):

$$\text{Cor}(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_x \sigma_y}$$

2.6 Mudança de variáveis (Jacobiano)

Quando se transforma um par de variáveis (X_1, X_2) em $(Y_1, Y_2) = (g_1(X_1, X_2), g_2(X_1, X_2))$ de forma invertível, a densidade conjunta se transforma usando o **determinante Jacobiano**:

$$J(x_1, x_2) = \begin{vmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} \end{vmatrix}, \quad f_{Y_1, Y_2}(y_1, y_2) = f_{X_1, X_2}(x_1, x_2) |J|^{-1}$$

Intuitivamente, o Jacobiano mede o “fator de distorção de área/volume” que a transformação provoca — é preciso compensar essa distorção para que a densidade continue integrando 1. O caso 1-D mais simples ilustra a ideia: se $Y = aX + b$, então $f_Y(y) = \frac{1}{|a|} f_X\left(\frac{y-b}{a}\right)$.

Se a densidade conjunta se **fatora** em uma parte dependente só de x e outra só de y , então X e Y são independentes.

2.7 Três desigualdades fundamentais

Estas desigualdades reaparecem repetidamente em Teoria da Informação (por exemplo, na prova de que a divergência KL nunca é negativa — Capítulo 7).

Desigualdade de Jensen

Para uma função **convexa** g e variável aleatória X com esperança finita:

$$E[g(X)] \geq g(E[X])$$

Nota de correção: alguns slides do curso escrevem essa desigualdade com o sinal invertido ($\leq$) — isso corresponderia ao caso de função **côncava**. A forma padrão para função convexa (ex.: $g(x) = x^2$, $g(x) = -\log x$) é $E[g(X)] \geq g(E[X])$; para função côncava (ex.: $g(x) = \log x$), vale o oposto: $E[g(X)] \leq g(E[X])$.

Intuição geométrica: para uma função convexa, existe sempre uma reta tangente $h(x) = ax + b$ no ponto $(E[X], g(E[X]))$ tal que $h(x) \leq g(x)$ para todo x (a curva fica sempre “acima” de suas tangentes). Tomando esperança dos dois lados: $E[h(X)] \leq E[g(X)]$. Como h é linear, $E[h(X)] = h(E[X]) = g(E[X])$ — daí a desigualdade.

Desigualdade de Markov

Para $X \geq 0$ e $a > 0$:

$$P(X \geq a) \leq \frac{E[X]}{a}$$

Só exige que $E[X]$ seja finito — é um limite “grosseiro” (mas válido para qualquer distribuição) da probabilidade de cauda, usando apenas a média.

Desigualdade de Chebyshev

Para X com média μ e variância σ^2 finitas, $k > 0$:

$$P(|X - \mu| \geq k) \leq \frac{\sigma^2}{k^2}$$

Demonstração: aplica-se Markov à variável não-negativa $(X - \mu)^2$ com $a = k^2$: $P((X - \mu)^2 \geq k^2) \leq \frac{E[(X - \mu)^2]}{k^2} = \frac{\sigma^2}{k^2}$; como $(X - \mu)^2 \geq k^2 \iff |X - \mu| \geq k$, chega-se ao resultado. Exemplo prático: com $k = 2\sigma$, a probabilidade de estar a mais de 2 desvios-padrão da média é no máximo $1/4 = 25\%$, para **qualquer** distribuição.

Importância conjunta: Markov e Chebyshev são “distribution-free” — não exigem conhecer a forma da distribuição, apenas média (Markov) ou média e variância (Chebyshev). Por isso são conservadoras (os limites que fornecem são folgados), mas extremamente úteis para provar teoremas gerais (ex.: Lei dos Grandes Números) sem hipóteses fortes sobre a distribuição.

Capítulo 3 — Estatística Descritiva e Pré-processamento de Dados

Este capítulo aplica os conceitos de probabilidade a **conjuntos de dados observados** (amostras), com foco no pré-processamento — etapa 2 do pipeline do Capítulo 1.

3.1 Medidas de posição

Para um conjunto de valores $\vec{v} = \{v_1, \dots, v_n\}$:

Média: $\mu = \frac{1}{n} \sum_{i=1}^n v_i$

Mediana: valor central da sequência ordenada v' :

$$M = \begin{cases} v'_{(n+1)/2}, & n \text{ ímpar} \\ \frac{1}{2} (v'_{n/2} + v'_{n/2+1}), & n \text{ par} \end{cases}$$

Moda: valor mais frequente.

Exemplo resolvido (dados do curso): quantidades observadas $\{29, 30, 32, 65, 65, 65, 25, 25, 90\}$. Ordenando: 25, 25, 29, 30, 32, 65, 65, 65, 90. Resulta em $\mu = 47,33$; $M = 32$; Moda = 65.

3.2 Medidas de dispersão

Amplitude: $a(\vec{v}) = \max(\vec{v}) - \min(\vec{v})$

Variância (amostral): $\sigma^2 = \frac{1}{n} \sum_{i=1}^n (v_i - \mu)^2$; **desvio padrão:** $\sigma = \sqrt{\sigma^2}$.

Coefficiente de variação de Pearson (dispersão relativa, útil para comparar distribuições em escalas diferentes):

$$CV(\vec{v}) = \frac{\sigma(\vec{v})}{\mu(\vec{v})}$$

Exemplo (usando o mesmo conjunto acima): $\sigma^2 \approx 553,6$, $\sigma \approx 23,5$, $CV \approx 0,5$ — a dispersão é da ordem de metade da média, indicando bastante heterogeneidade nos dados.

3.3 Distribuição e frequência

Frequência relativa de uma faixa γ : $fr(\gamma) = \frac{f(\gamma)}{\sum_i f(v_i)} \times 100$.

Frequência acumulada: soma das frequências relativas até a faixa corrente: $far(\gamma_k) = \sum_{i=1}^k fr(\gamma_i)$.

Se ordenada de forma decrescente, a distribuição de frequência gera o clássico **Diagrama de Pareto**, usado como indicador de qualidade/priorização (poucos itens concentram a maior parte da frequência/impacto — o famoso “princípio 80/20”).

3.4 Quartis e percentis

Os **quartis** dividem o conjunto ordenado em 4 grupos de aproximadamente 25% cada: Q_1 (25º percentil), Q_2 (50º percentil = mediana), Q_3 (75º percentil). A representação visual (**boxplot**) permite ver de uma só vez posição, dispersão, simetria, caudas e **outliers** (valores atípicos, tipicamente fora do intervalo $[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR]$, onde $IQR = Q_3 - Q_1$).

3.5 Correlação amostral

$$r_{\vec{v}_1, \vec{v}_2} = \frac{\sum_i (v_{1i} - \mu_1)(v_{2i} - \mu_2)}{\sqrt{\sum_i (v_{1i} - \mu_1)^2} \sqrt{\sum_i (v_{2i} - \mu_2)^2}} = \frac{cov(\vec{v}_1, \vec{v}_2)}{\sigma_1 \sigma_2}, \quad -1 \leq r \leq 1$$

Casos de interpretação: - **Correlação positiva**: as variáveis crescem/decrescem juntas. - **Correlação negativa**: uma cresce quando a outra decresce. - **Correlação nula**: sem padrão linear identificável (pode haver dependência não-linear!).

Esta é a versão *amostral* (soma sobre observações) da correlação populacional/teórica $Cor(X, Y)$ do Capítulo 2 — formalmente equivalentes, trocando somatório por valor esperado.

3.6 Recomendações de uso de métricas

Medida	Uso recomendado	Observação
Moda / Frequência	Dados categóricos	—
Percentil	Dados contínuos	—
Média	Localização	Sensível a <i>outliers</i>
Mediana	Localização	Robusta a <i>outliers</i> — preferível na presença deles
Variância / desvio padrão	Dispersão	As mais usadas

Uma prática comum para reduzir o impacto de valores extremos é a **média aparada** (*trimmed mean*): remover os valores mínimo e máximo antes de calcular a média, ou simplesmente preferir a mediana.

3.7 Transformação de dados

Antes da análise, dados heterogêneos costumam precisar de ajustes:

- **Discretização:** numérico $\rightarrow$ categórico (ex.: idade $\rightarrow$ faixa etária).
- **Codificação (encoding):** categórico $\rightarrow$ numérico (necessário para muitos algoritmos).
- **Agrupamento por frequência:** reduzir categorias raras a um grupo “outros”.
- **Normalização:** ver seção seguinte — fundamental quando atributos têm escalas muito diferentes.

3.8 Normalização de dados

Motivação: se um atributo varia entre 0 e 1 e outro entre 0 e 1.000.000, algoritmos baseados em distância (KNN, k-means, redes neurais) serão dominados pelo atributo de maior escala, mesmo que ele não seja mais “importante”. A normalização equaliza as escalas.

Método 1 — Norma constante (normalização local)

Divide cada vetor pela própria norma euclidiana, tornando-o unitário:

$$\vec{x}^* = \frac{\vec{x}}{\|\vec{x}\|}$$

Exemplo: $\vec{x} = (\sqrt{3}, 3, -2)^T$, $\|\vec{x}\| = \sqrt{3+9+4} = \sqrt{16} = 4$, logo $\vec{x}^* = (\sqrt{3}/4, 3/4, -1/2)^T$. Preserva a **direção** do vetor, altera apenas o comprimento; depende só das próprias componentes do vetor (por isso “local”).

Método 2 — Mudança de escala (min-max, normalização global)

Aplicada variável a variável, exigindo x_{min} e x_{max} dessa variável. Para o intervalo $[0, 1]$:

$$x^* = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Para o intervalo $[-1, +1]$:

$$x^* = 2 \cdot \frac{x - x_{min}}{x_{max} - x_{min}} - 1$$

Método 3 — Padronização (z-score)

Aplicada variável a variável, resultando em **média 0 e variância 1**:

$$x^* = \frac{x - \mu}{\sigma_x}, \quad \sigma_x = \sqrt{\frac{\sum_i (x_i - \mu)^2}{N - 1}} \text{ (desvio-padrão amostral, com correção de Bessel } N - 1)$$

Normalização como transformação linear e suas propriedades

Todos os três métodos são casos particulares de uma transformação afim $x^* = ax + b$. Isso traz duas propriedades importantes:

1. **Preserva a forma da distribuição:** como $Y = aX + b$ é linear, se X era gaussiana, Y continua gaussiana — apenas os parâmetros (média, variância) mudam, não a família da distribuição. Formalmente, $F_Y(y) = F_X\left(\frac{y-b}{a}\right)$ para $a > 0$, uma simples reparametrização.
 2. **Preserva a correlação entre variáveis:** como cada variável é normalizada isoladamente (usando apenas suas próprias estatísticas — min, max, média, desvio), a correlação entre duas variáveis quaisquer **não muda**. As variáveis normalizadas tornam-se **adimensionais**, facilitando a comparação.
-

Capítulo 4 — Distribuições de Probabilidade Contínuas

Este capítulo cataloga as distribuições contínuas mais usadas no curso — ferramenta essencial para modelar fontes de informação, ruído de canal (Capítulo 10) e técnicas de predição (Capítulo 5).

Distribuição	PDF	$E[X]$	$Var(X)$	Uso típico
Pareto (θ, α)	$\frac{\alpha\theta^\alpha}{x^{\alpha+1}}, x \geq \theta$	$\frac{\alpha\theta}{\alpha-1}$ (se $\alpha > 1$)	existe se $\alpha > 2$	Fenômenos de cauda pesada
Exponencial (λ)	$\lambda e^{-\lambda x}, x \geq 0$	$1/\lambda$	$1/\lambda^2$	Tempo até um evento
Normal (μ, σ^2)	$\frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$	μ	σ^2	Erros de medição, ruído, TLC
Log-normal	$Y = e^X, X \sim N(\mu, \sigma^2)$	$e^{\mu+\sigma^2/2}$	$(e^{\sigma^2} - 1)e^{2\mu+\sigma^2}$	Variáveis econômicas (renda)
Gama (α, λ)	$\frac{\lambda e^{-\lambda x} (\lambda x)^{\alpha-1}}{\Gamma(\alpha)}$	α/λ	α/λ^2	Tempo até n eventos (Poisson)
Beta (a, b)	$\frac{x^{a-1}(1-x)^{b-1}}{B(a,b)}, 0 < x < 1$	$\frac{a}{a+b}$	$\frac{ab}{(a+b)^2(a+b+1)}$	Fenômenos limitados a um intervalo
Weibull (v, α, β)	ver abaixo	—	—	Confiabilidade/vida útil
Uniforme $[a, b]$	$\frac{1}{b-a}$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	Erros pequenos; prior não-informativo

4.1 Distribuição Exponencial e a propriedade de falta de memória

$$f(x) = \lambda e^{-\lambda x} \quad (x \geq 0), \quad F(a) = 1 - e^{-\lambda a}$$

Propriedade da falta de memória: $P(X > s+t \mid X > s) = P(X > t)$. Se X é o tempo de vida de um equipamento, a chance de durar mais t anos, dado que já durou s anos, é igual à chance de um equipamento **novo** durar t anos — o equipamento “não envelhece” sob esse modelo.

4.2 Distribuição Gama e sua relação com Exponencial, Erlang e Qui-quadrado

A função Gama $\Gamma(\alpha) = \int_0^\infty e^{-y} y^{\alpha-1} dy$ satisfaz $\Gamma(\alpha) = (\alpha - 1)\Gamma(\alpha - 1)$ e, para n inteiro, $\Gamma(n) = (n - 1)!$.

Parâmetros	Nome
$\alpha = 1$	Exponencial
$\alpha = k$ inteiro	Erlang de ordem k (tempo até k eventos de Poisson)
$\alpha = n/2, \lambda = 1/2$	Qui-quadrado com n graus de liberdade

4.3 Relação Gama–Beta

Se $X \sim Gama(\alpha, \lambda)$ e $Y \sim Gama(\beta, \lambda)$ são independentes, definindo $U = X + Y$ e $V = X/(X + Y)$, então **U e V são independentes**, com $U \sim Gama(\alpha + \beta, \lambda)$ e $V \sim Beta(\alpha, \beta)$ — um resultado elegante que se demonstra via mudança de variáveis com Jacobiano $J(x, y) = -\frac{1}{x+y}$ (ver Capítulo 2, seção 2.6).

4.4 Weibull como distribuição “camaleão”

$$F(x) = 1 - \exp \left\{ - \left(\frac{x - v}{\alpha} \right)^\beta \right\}, \quad x > v$$

O parâmetro de forma β controla a que outra distribuição a Weibull se aproxima:

β	Aproxima
$\beta = 1$	Exponencial (equivalência exata)
$1 < \beta < 3,6$	Log-normal
$\beta \approx 3,6$	Normal

Por essa flexibilidade, é amplamente usada em análise de confiabilidade/vida útil de componentes.

4.5 Distribuição Normal e o Teorema do Limite Central

Introduzida por DeMoivre (1733) como aproximação da distribuição binomial para n grande. É a base do **Teorema do Limite Central** (a soma de muitas variáveis aleatórias independentes, com variância finita, tende a uma distribuição Normal, independentemente da distribuição original de cada termo) — motivo pelo qual aparece tão frequentemente como modelo de ruído em sistemas de comunicação (Capítulo 10) e de erro de medição.

Capítulo 5 — Inferência, Processos Estocásticos e Técnicas de Predição

Este capítulo cobre o ferramental estatístico de predição/inferência usado como pano de fundo em aplicações de Big Data e Machine Learning (Parte III).

5.1 Teorema de Bayes

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}, \quad P(X) = \sum_Y P(X, Y)$$

Permite atualizar a crença sobre Y (probabilidade *a posteriori*) após observar evidência X , a partir de uma crença inicial $P(Y)$ (*a priori*) e do modelo de verossimilhança $P(X|Y)$.

5.2 Processos estocásticos e Cadeias de Markov

Um **processo estocástico** é uma família de variáveis aleatórias indexadas por um parâmetro (tipicamente o tempo): $X(t, \omega)$.

Propriedade de Markov: dado o presente, o futuro é independente do passado:

$$P(X_{n+1} = x_{n+1} \mid X_0, \dots, X_n) = P(X_{n+1} = x_{n+1} \mid X_n)$$

A matriz de transição $P = (p_{ij})$, com $p_{ij} = P(X_{n+1} = j \mid X_n = i)$, é estocástica ($\sum_j p_{ij} = 1$). A **equação de Chapman-Kolmogorov** permite compor transições de múltiplos passos: $P_{ij}(m+n) = \sum_k P_{ik}(m)P_{kj}(n)$, ou seja, $P^{(n)} = P^n$ (a matriz de transição em n passos é a n -ésima potência da matriz de um passo).

Exemplo resolvido — previsão do tempo (0 = chove, 1 = não chove):

$$P = \begin{pmatrix} 0,6 & 0,4 \\ 0,2 & 0,8 \end{pmatrix} \Rightarrow P^2 = \begin{pmatrix} 0,44 & 0,56 \\ 0,28 & 0,72 \end{pmatrix} \Rightarrow P^4 = \begin{pmatrix} 0,3504 & 0,6496 \\ 0,3248 & 0,6752 \end{pmatrix}$$

Logo, partindo de um dia chuvoso, a chance de chover novamente daqui a 4 dias é $p_{00}^{(4)} = 0,3504$.

Cadeia ergódica: se toda transição $P_{ij}^{(m)} > 0$ para algum m , a cadeia converge para uma **distribuição estacionária** π que não depende do estado inicial.

5.3 Teoria da decisão

Para prever uma saída (contínua = regressão; categórica = classificação) a partir de uma entrada x , três abordagens são possíveis:

1. **Modelo generativo:** estima $P(x|C_k)$ e $P(C_k)$ separadamente e aplica Bayes: $P(C_k|x) = \frac{P(x|C_k)P(C_k)}{P(x)}$. Mais custoso, mas permite gerar dados sintéticos e detectar *outliers*.
2. **Modelo discriminativo:** estima diretamente $P(C_k|x)$, sem passar pela distribuição conjunta.
3. **Função discriminante:** aprende diretamente uma função $f(x)$ que mapeia x a uma classe, sem calcular probabilidades — mais simples e barato, mas perde informação útil (ex.: para rejeitar decisões de baixa confiança).

A **regra de decisão de erro mínimo:** classificar x na classe de maior $P(C_k|x)$ minimiza a probabilidade de erro. Quando os erros têm custos diferentes, usa-se uma **matriz de perda** L_{kj} e minimiza-se a **perda esperada** em vez da taxa de erro bruta.

5.4 Técnicas de predição (visão panorâmica)

Técnica	Ideia central
Mínimos Quadrados	$\hat{\beta} = (X^T X)^{-1} X^T Y$ — minimiza a soma dos erros ao quadrado
KNN (k-vizinhos)	Prediz pela média dos k pontos mais próximos; sensível à escala (daí a importância da normalização, Cap. 3)
MLE (Máxima Verossimilhança)	Escolhe os parâmetros que tornam os dados observados mais prováveis; não-viesado e consistente, porém custoso
Regularização (Ridge/Lasso)	Penaliza coeficientes grandes; Ridge (penalidade $L2$) encolhe coeficientes, Lasso (penalidade $L1$) zera coeficientes (seleção de variáveis)
LARS	Variante eficiente do <i>forward stepwise</i> , intimamente ligada ao Lasso
Regressão local / Kernel	Ajusta um modelo separado para cada ponto-alvo, ponderando vizinhos por uma função kernel
Regressão Bayesiana	Trata os parâmetros como variáveis aleatórias com prior/posterior Gaussianos
Monte Carlo / Importance Sampling	Aproxima integrais difíceis (como $E[f]$) por amostragem
Modelos de Mistura Finita	População = combinação de K subgrupos com distribuições próprias — base teórica do <i>clustering</i> probabilístico (GMM)

Essas técnicas reaparecem, em versão simplificada, no Capítulo 14 (treinamento de redes neurais) — o mínimo quadrado, por exemplo, é exatamente a função de perda (MSE) usada para treinar um

modelo de regressão linear via gradiente descendente.

PARTE II — TEORIA DA INFORMAÇÃO CLÁSSICA (SHANNON)

Capítulo 6 — Quantidade de Informação e Entropia

Chegamos ao núcleo conceitual da disciplina. A pergunta central da Teoria da Informação, formulada por **Claude Shannon** em seu artigo seminal de 1948 (“A Mathematical Theory of Communication”), é:

Quanta informação uma variável aleatória contém, quando observamos um valor específico dela?

6.1 Autoinformação (informação de um evento)

Considere uma fonte discreta com alfabeto $A = \{a_0, a_1, \dots, a_{K-1}\}$ e distribuição de probabilidade $P(A = a_k) = p_k$, com $\sum_k p_k = 1$.

A informação pode ser entendida como um **nível de surpresa**: um evento improvável nos surpreende mais (carrega mais informação); um evento certo não surpreende ninguém (carrega informação zero). Isso motiva a definição:

$$I(a_k) = \log_{\alpha} \left(\frac{1}{p_k} \right) = -\log_{\alpha}(p_k)$$

A base α do logaritmo define a unidade: $\alpha = 2 \rightarrow$ **bits**; $\alpha = e \rightarrow$ **nats**; $\alpha = 10 \rightarrow$ **hartleys/dits**. Salvo indicação contrária, este material usa $\log_2$ (bits).

Propriedades da autoinformação: 1. $I(a_k) = 0$ se $p_k = 1$ (evento certo não traz novidade). 2. $I(a_k) \geq 0$ para $0 \leq p_k \leq 1$ (informação nunca é negativa). 3. $I(a_k) > I(a_i)$ se $p_k < p_i$ (eventos mais raros carregam mais informação). 4. $I(a_k a_i) = I(a_k) + I(a_i)$ se a_k e a_i são independentes (informação de eventos independentes é **aditiva**).

A propriedade 4 é justamente a razão de usarmos o **logaritmo**: para que a informação de eventos conjuntos independentes seja a soma das informações individuais, é necessário que $I(p_k \cdot p_i) = I(p_k) + I(p_i)$ — e essa é exatamente a propriedade fundamental do logaritmo, $\log(xy) = \log x + \log y$.

A curva $I(p) = -\log_2(p)$ no intervalo $(0, 1]$ é estritamente decrescente e convexa: tende a infinito quando $p \rightarrow 0$ e vale exatamente 0 quando $p = 1$.

6.2 Entropia de Shannon

A **entropia** é a média (valor esperado) da autoinformação sobre todos os símbolos da fonte:

$$H(A) = \sum_k p_k \cdot I(a_k) = - \sum_k p_k \log_{\alpha}(p_k) = -E\{\log(p_A(a))\}$$

Interpretação: $H(A)$ mede a **quantidade média de informação (ou incerteza) por símbolo** emitido pela fonte. Também pode ser lida como o “custo médio em bits” (quando $\alpha = 2$) necessário para representar cada símbolo — essa leitura é o que conecta entropia a compressão de dados (Capítulo 9).

Propriedades:

$$0 \leq H(A) \leq \log_{\alpha} K$$

onde $K = |A|$ é o tamanho do alfabeto.

- **Mínimo** ($H = 0$): quando um símbolo tem $p_k = 1$ e os demais têm probabilidade zero — não há incerteza, a fonte é totalmente determinística.
- **Máximo** ($H = \log_{\alpha} K$): quando todos os símbolos são **equiprováveis** ($p_k = 1/K$) — máxima incerteza possível.

Caso binário: para uma fonte com dois símbolos, p_0 e $p_1 = 1 - p_0$:

$$H(A) = -p_0 \log_2 p_0 - (1 - p_0) \log_2 (1 - p_0) \equiv H_b(p_0)$$

O gráfico dessa **função de entropia binária** é uma curva côncava em forma de “sino”, valendo 0 em $p_0 = 0$ e $p_0 = 1$, e atingindo o valor máximo de **1 bit** exatamente em $p_0 = 0,5$. Interpretação: uma moeda “viciada” (p_0 perto de 0 ou 1) tem pouca incerteza (poucos “surpresas”); uma moeda “honesta” ($p_0 = 0,5$) tem incerteza máxima.

6.3 Extensão de uma fonte discreta sem memória

Se agruparmos n símbolos consecutivos de uma fonte sem memória em um “superssímbolo”, o alfabeto estendido A^n tem K^n blocos possíveis. Como os símbolos originais são estatisticamente independentes:

$$P(s[A^n]) = \prod_{i=1}^n P(s_i[A]) \implies H(A^n) = n \cdot H(A)$$

Esse resultado, aparentemente simples, é a base teórica para a codificação por blocos e para o **Primeiro Teorema de Shannon** (Capítulo 9): quanto maior o bloco, mais próxima a taxa de codificação pode chegar da entropia teórica da fonte.

Capítulo 7 — Entropia Conjunta, Condicional, Divergência KL e Informação Mútua

7.1 Entropia conjunta

Estende a entropia para duas variáveis aleatórias, medindo a incerteza total de um sistema composto:

$$H(A, B) = - \sum_{a \in A} \sum_{b \in B} p(a, b) \log[p(a, b)] = -E\{\log[p(a, b)]\}$$

Exemplo (moeda honesta + dado honesto de 6 faces, independentes): há 12 combinações equiprováveis, cada uma com probabilidade $1/12$:

$$H(X, Y) = - \sum_{i=1}^{12} \frac{1}{12} \log_2 \frac{1}{12} = \log_2(12) \approx 3,585 \text{ bits}$$

7.2 Entropia condicional

Mede a incerteza **remanescente** sobre A depois de conhecer B :

$$H(A|B) = \sum_b p(b) H(A|B=b) = - \sum_b \sum_a p(a, b) \log[p(a|b)] = -E\{\log[p(a|b)]\}$$

No exemplo moeda + dado: como as variáveis são independentes, conhecer o dado não ajuda a prever a moeda: $H(\text{moeda} | \text{dado}) = H(\text{moeda}) = 1 \text{ bit}$.

7.3 Regra da cadeia

$$H(A, B) = H(A) + H(B|A) = H(B) + H(A|B)$$

A incerteza total de um sistema conjunto é a incerteza de uma parte mais a incerteza *remanescente* da outra parte, dado que a primeira já é conhecida. Generaliza-se para n variáveis:

$$H(A_1, \dots, A_N) = \sum_{i=1}^N H(A_i | A_{i-1}, \dots, A_1)$$

7.4 Divergência de Kullback-Leibler (entropia relativa / entropia cruzada)

Mede a “distância” (não simétrica) entre duas distribuições p e g — mais precisamente, a **ineficiência** de assumir que uma variável segue $p(x)$ quando a verdadeira distribuição é $g(x)$:

$$D(p\|g) = \sum_x p(x) \log \left(\frac{p(x)}{g(x)} \right) = E_p \left\{ \log \frac{p(x)}{g(x)} \right\}$$

Propriedades: 1. $D(p\|g) \geq 0$, com igualdade **se e somente se** $p = g$ (consequência direta da desigualdade de Jensen aplicada à função convexa $-\log(\cdot)$ — ver Capítulo 2). 2. **Não é simétrica:** $D(p\|g) \neq D(g\|p)$ em geral — por isso, apesar de ser chamada informalmente de “distância”, **não é uma métrica** no sentido matemático estrito (não satisfaz simetria nem, em geral, a desigualdade triangular). 3. Invariante a permutação, escalonamento de amplitude e transformações monotônicas não-lineares das componentes.

A KLD é amplamente usada em Machine Learning como função de perda (ex.: *cross-entropy loss* em classificação, e como componente central em modelos generativos como VAEs).

7.5 Informação Mútua

Definida como a divergência KL entre a distribuição conjunta e o produto das marginais:

$$I(A, B) = \sum_{a,b} p(a, b) \log \left(\frac{p(a, b)}{p(a)p(b)} \right) = D(p(a, b) \parallel p(a)p(b))$$

Relações fundamentais:

$$I(A, B) = H(A) - H(A|B) = H(B) - H(B|A) = H(A) + H(B) - H(A, B)$$

Ou seja, a informação mútua mede **quanto conhecer B reduz a incerteza sobre A** (e vice-versa, por simetria). Se A e B são independentes, $I(A, B) = 0$; quanto mais dependentes, maior $I(A, B)$. Como caso particular, $I(A, A) = H(A)$ (conhecer A elimina toda a incerteza sobre si mesma, pois $H(A|A) = 0$).

Diagrama de Venn (forma mais intuitiva de visualizar tudo isso): imagine dois círculos sobrepostos representando $H(A)$ e $H(B)$. A **união** dos dois círculos é $H(A, B)$; a **interseção** é a informação mútua $I(A, B)$; as partes exclusivas de cada círculo são $H(A|B)$ e $H(B|A)$, respectivamente.

7.6 Ganho de Informação e árvores de decisão

Em Machine Learning, a informação mútua reaparece com outro nome: **Ganho de Informação** (*Information Gain*):

$$IG(Y|X) = H(Y) - H(Y|X) \equiv I(X, Y)$$

É o critério-chave para escolher, em algoritmos de árvore de decisão (ex.: C4.5), qual atributo X usar para dividir os dados: escolhe-se o atributo que **mais reduz** a incerteza sobre a variável-alvo Y . **Limitação:** o ganho de informação tende a favorecer atributos com muitos valores distintos (mesmo que irrelevantes) — por isso existem alternativas como **Gain Ratio** e **Índice de Gini** (usado no algoritmo CART).

Capítulo 8 — Entropia Diferencial e Generalizações da Entropia

8.1 Entropia diferencial

Quando a variável é **contínua**, a entropia recebe o nome de **entropia diferencial**:

$$H(X) = - \int_S p_X(x) \log[p_X(x)] dx$$

onde S é o suporte da variável. **Diferença importante em relação ao caso discreto**: a entropia diferencial **pode ser negativa** — não existe o limite inferior $H \geq 0$ que vale no caso discreto, pois a densidade $p_X(x)$ pode assumir valores maiores que 1.

Exemplo resolvido — entropia diferencial da Normal ($X \sim N(0, \sigma^2)$, em nats):

$$H[p_X(x)] = - \int \phi(x) \ln[\phi(x)] dx = \frac{E[X^2]}{2\sigma^2} + \frac{1}{2} \ln(2\pi\sigma^2) = \frac{1}{2} + \frac{1}{2} \ln(2\pi\sigma^2) = \frac{1}{2} \ln(2\pi e\sigma^2)$$

O resultado depende **apenas da variância** — quanto maior a dispersão, maior a entropia diferencial. Este é o resultado clássico que mostra que, entre todas as distribuições contínuas com variância fixa, a **Normal é a que maximiza a entropia diferencial** (máxima incerteza compatível com uma dada variância) — resultado que será usado diretamente na dedução da capacidade de canal gaussiano (Capítulo 10).

Propriedades da entropia diferencial (diferentes do caso discreto): - $H(X + c) = H(X)$ — deslocamento não altera a entropia. - $H(cX) = H(X) + \log|c|$ — escalonamento por c altera a entropia pelo log do valor absoluto de c . - $H(CX) = H(X) + \log|\det C|$ para transformações matriciais (vetores).

Ou seja, ao contrário da entropia discreta, a entropia diferencial **não é invariante a mudanças de escala** — deve ser interpretada como medida relativa/comparativa, não como “quantidade absoluta de informação” no mesmo sentido do caso discreto.

8.2 Propriedades gerais (válidas nos dois casos)

1. $D(p||g) \geq 0$ sempre.
2. $I(X, Y) \geq 0$, com igualdade sse independentes.

3. $H(X|Y) \leq H(X)$, com igualdade sse independentes (condicionar nunca aumenta, em média, a incerteza).

8.3 Outras definições de entropia

A entropia de Shannon é apenas um membro de uma família mais ampla de medidas de incerteza/diversidade:

Entropia de Rényi

Generaliza Shannon por um parâmetro de ordem α ($\alpha > 0$, $\alpha \neq 1$):

$$H_\alpha(X) = \frac{1}{1-\alpha} \log \left(\sum_i p_i^\alpha \right)$$

Casos particulares importantes: - $\alpha \rightarrow 0$: **Entropia de Hartley**, $H_0 = \log(N)$. - $\alpha \rightarrow 1$: recupera exatamente a **entropia de Shannon**. - $\alpha = 2$: $H_2(X) = -\log(\sum_i p_i^2)$ — particularmente conveniente computacionalmente (ver estimador de Parzen, Capítulo 9). - $\alpha \rightarrow \infty$: **Min-entropia**, $H_\infty = -\log(\sup_i p_i)$ — usada em criptografia como medida de “pior caso” de aleatoriedade de uma chave.

Valores pequenos de α dão mais peso a eventos raros (diversidade); valores grandes dão mais peso ao evento mais provável (dominância). Usada em ecologia (índices de diversidade), segurança (min-entropia) e física estatística.

Entropia de Boltzmann-Gibbs (origem física do nome “entropia”)

$$H = -k_B \sum_\alpha p_\alpha \log p_\alpha$$

onde k_B é a constante de Boltzmann e p_α é a probabilidade do sistema estar no microestado α . Matematicamente idêntica à entropia de Shannon (a menos da constante k_B) — foi, de fato, a inspiração histórica (por sugestão de John von Neumann) para Shannon batizar seu conceito de “entropia”.

Entropia de Tsallis

Generaliza Boltzmann-Gibbs por um parâmetro q :

$$H_q(p) = \frac{1}{q-1} \left(1 - \sum p^q \right)$$

Recupera Boltzmann-Gibbs quando $q \rightarrow 1$. Usada para sistemas com correlações de longo alcance (estatísticas não-extensivas).

Entropia de von Neumann (domínio quântico)

$$H(\rho) = \text{tr}[\rho \log(\rho)]$$

onde ρ é a **matriz densidade** (Hermitiana, positiva, traço 1) que descreve um estado quântico. É o análogo direto da entropia de Shannon no domínio quântico: em vez de uma distribuição de probabilidade sobre estados clássicos, usam-se os autovalores de ρ como se fossem probabilidades.

Entropia Espectral

Aplica a fórmula de Shannon à **densidade espectral** de um sinal em vez de a uma distribuição de probabilidade:

$$H_{sp}(P) = - \sum_{i=f_l}^{f_h} P_i \log(P_i)$$

Aplicação notável: medir profundidade de sedação/anestesia via EEG — um espectro mais “espalhado” (maior entropia espectral) indica maior atividade/consciência; um espectro concentrado indica sedação profunda.

Capítulo 9 — Codificação de Fonte e Compressão de Dados

9.1 O problema da codificação de fonte

Codificação de fonte: representar os símbolos de uma fonte de forma eficiente. Princípio universal: **símbolos mais frequentes recebem palavras-código menores; símbolos raros recebem palavras-código maiores** (o código Morse já usava essa ideia no século XIX: a letra mais frequente do inglês, ‘E’, é um único ponto ‘.’, enquanto ‘Q’, rara, é ‘- . - .’).

Formalizando: fonte com K símbolos, probabilidades p_k , palavras-código binárias de comprimento l_k . O **comprimento médio**:

$$L = \sum_k p_k \cdot l_k$$

A **eficiência de codificação** é $\eta = L_{min}/L$ (idealmente $\eta \rightarrow 1$).

9.2 O Primeiro Teorema de Shannon (Teorema da Codificação de Fonte)

$$L \geq H(A)$$

A entropia da fonte é o **limite fundamental** — não existe codificação sem perdas cujo comprimento médio fique abaixo da entropia. O limite é alcançável ($L_{min} = H(A)$) apenas em casos especiais (ver Kraft-McMillan abaixo). Combinando com a extensão de fonte do Capítulo 6 ($H(A^n) = nH(A)$), tem-se o resultado assintótico:

$$\lim_{n \rightarrow \infty} \frac{L_n}{n} = H(A)$$

— codificando blocos cada vez maiores de símbolos, a taxa de codificação por símbolo se aproxima arbitrariamente da entropia, ao custo de uma complexidade de decodificação crescente.

9.3 Códigos prefixo e a Inequação de Kraft-McMillan

Um **código prefixo** é aquele em que nenhuma palavra-código é prefixo de outra. Isso garante que o código seja **unicamente decodificável** e, mais que isso, **instantâneo** (decodificável símbolo a símbolo, sem esperar bits futuros) — a decodificação corresponde a percorrer uma árvore binária até uma folha.

Inequação de Kraft-McMillan: os comprimentos l_k de um código prefixo devem satisfazer

$$\sum_k 2^{-l_k} \leq 1$$

e, reciprocamente, se um conjunto de comprimentos satisfaz essa desigualdade, **existe** um código prefixo construível com esses comprimentos.

Limites do comprimento médio: para qualquer código prefixo de uma fonte com entropia $H(A)$:

$$H(A) \leq L < H(A) + 1$$

A igualdade à esquerda ocorre quando $p_k = 2^{-l_k}$ exatamente para todo k (código “absolutamente ótimo”).

9.4 Codificação de Huffman

Ideia: associar a cada símbolo uma palavra-código de comprimento aproximadamente igual à sua autoinformação $-\log_2 p_k$, aproximando o limite teórico da entropia.

Algoritmo (passo a passo)

1. Liste os símbolos em ordem **decrescente** de probabilidade. Atribua os rótulos ‘0’ e ‘1’ aos dois símbolos de **menor** probabilidade.
2. **Combine** esses dois símbolos em um novo símbolo, com probabilidade igual à soma das duas. Reordene a lista.
3. Repita até restarem apenas dois símbolos.
4. Reconstrua cada palavra-código percorrendo o processo **de trás para frente** (retropropagação), concatenando os bits atribuídos em cada fusão pela qual o símbolo original passou.

Exemplo resolvido

Fonte com distribuição:

Símbolo	a_0	a_1	a_2	a_3	a_4
Probabilidade	0,4	0,2	0,2	0,1	0,1

Entropia da fonte: $H(A) = 2,12193$ bits/símbolo.

Uma construção possível de Huffman: fundir primeiro os dois menores ($a_3 + a_4 = 0,2$); reordenar $\{a_0:0,4, [a_1:0,2, a_2:0,2, a_3a_4:0,2]\}$; fundir dois dos 0,2 (ex.: $a_2 + a_3a_4 = 0,4$); reordenar $\{a_0:0,4, a_1:0,2, a_2a_3a_4:0,4\}$; fundir a_1 com a_0 ou com $a_2a_3a_4$... Uma codificação consistente resultante:

Símbolo	Código	l_k
a_0	1	1
a_1	01	2
a_2	000	3

Símbolo	Código	l_k
a_3	0010	4
a_4	0011	4

$$L = 0,4(1) + 0,2(2) + 0,2(3) + 0,1(4) + 0,1(4) = 0,4 + 0,4 + 0,6 + 0,4 + 0,4 = 2,2$$

Confirma-se $H(A) \leq L < H(A) + 1$ ($2,122 \leq 2,2 < 3,122$), com L apenas **3,67%** acima da entropia teórica.

Não-unicidade e limitações

O código de Huffman **não é único**: a atribuição de ‘0’/‘1’ em cada fusão é arbitrária, e empates de probabilidade podem ser resolvidos de formas diferentes. Diferentes execuções podem gerar códigos com o mesmo L ótimo, mas variância diferente:

$$\sigma^2 = \sum_k p_k (l_k - L)^2$$

Entre soluções igualmente ótimas em L , prefere-se a de menor variância (tamanhos de palavra mais previsíveis, úteis para buffers).

Limitação principal: exige conhecimento *a priori* das probabilidades exatas da fonte — nem sempre disponível ou estável (fontes dinâmicas exigiriam reconstruir a árvore continuamente).

9.5 Codificação Lempel-Ziv-Welch (LZW)

Alternativa **adaptativa** (não exige estatísticas *a priori*) e de implementação mais simples que Huffman. Constrói um **dicionário dinâmico** de subsequências ainda não vistas.

Algoritmo de compressão

1. Inicialize o dicionário com todas as strings de comprimento 1 (tipicamente ASCII).
2. Encontre a string mais longa W já no dicionário que casa com a entrada atual.
3. Emita o **índice** de W na saída; avance o ponteiro de leitura.
4. Adicione ao dicionário a string “ W + próximo símbolo”.
5. Repita até consumir toda a entrada.

Exemplo (sequência 000101110010100101): dicionário inicial $\{0, 1\} \rightarrow$ identifica 00 (novo) $\rightarrow$ dicionário $\{0, 1, 00\} \rightarrow$ identifica 01 $\rightarrow$ dicionário $\{0, 1, 00, 01\} \rightarrow$ identifica 011 $\rightarrow$ e assim por diante.

Características: sem perdas; o dicionário não precisa ser transmitido junto com o arquivo (é **reconstruído simetricamente** durante a decodificação); usa tipicamente códigos de tamanho fixo (ex.: 12 bits) para índices, ao contrário do tamanho variável de Huffman; muito eficaz em dados com alta repetição de padrões (ex.: imagens GIF/TIFF).

9.6 Compressão com e sem perdas — visão geral

Tipo	Característica	Exemplos	Uso
Sem perdas (lossless)	Recupera os dados originais exatamente	Huffman, LZ77, LZ78, LZW	Texto, código-fonte, bancos de dados
Com perdas (lossy)	Descarta informação pouco perceptível	JPEG, MP3, MPEG	Imagem, áudio, vídeo

9.7 Estimando entropia a partir de dados reais

Na prática, a densidade de probabilidade $p_X(x)$ raramente é conhecida — precisa ser **estimada** a partir de amostras. Duas abordagens citadas no curso:

- **Expansão de Gram-Charlier/Edgeworth:** aproxima p_X como uma correção polinomial em torno de uma gaussiana, usando momentos/cumulantes de ordem superior. Funciona bem quando a densidade real está “próxima” de uma gaussiana; conecta-se ao conceito de **negentropia** ($N_G(p_X) = H(p_G) - H(p_X)$, sempre ≥ 0 , zero se e só se p_X é gaussiana — uma medida de “quão não-gaussiana” é a distribuição, usada por exemplo em Análise de Componentes Independentes, ICA).
- **Estimador de Parzen (kernel density estimation):** aproxima qualquer densidade por uma soma de funções kernel centradas nas amostras, $p_X(x) = \frac{1}{N} \sum_i K(x - x_i, \sigma_i I)$. É especialmente conveniente para a entropia de Rényi de ordem 2, pois a integral de p^2 se resolve analiticamente em uma soma dupla fechada sobre pares de amostras — a chamada “entropia quadrática de Rényi”, muito usada em *Information Theoretic Learning* (ITL).

Capítulo 10 — Capacidade de Canal, Ruído e o Limite de Shannon

10.1 O problema da codificação de canal

Enquanto a codificação de **fonte** (Capítulo 9) busca **remover** redundância (compressão), a codificação de **canal** busca **inserir** redundância controlada, para proteger a mensagem contra os efeitos do ruído.

Exemplo intuitivo: repetir cada bit três vezes ($0 \rightarrow 000$, $1 \rightarrow 111$) e, no receptor, decidir pelo bit majoritário — se apenas um dos três bits for corrompido, o erro é corrigido. Essa é a ideia por trás de qualquer código de canal, ainda que os esquemas práticos sejam mais sofisticados (Capítulo 11).

Códigos de bloco: a mensagem é dividida em blocos de k bits, cada um mapeado em um bloco de $n > k$ bits (com $n - k$ bits de redundância). A **taxa do código**:

$$r = \frac{k}{n}$$

10.2 Segundo Teorema de Shannon (Teorema da Codificação do Canal)

Seja uma fonte com entropia $H(A)$ emitindo um símbolo a cada T_s segundos, e um canal discreto sem memória com capacidade C (bits por uso), usado a cada T_c segundos.

Parte de existência: se

$$\frac{H(A)}{T_s} \leq \frac{C}{T_c}$$

existe um esquema de codificação tal que a informação pode ser transmitida e reconstruída com probabilidade de erro **arbitrariamente pequena**.

Parte de impossibilidade (recíproca): se a desigualdade for invertida, **não** é possível atingir erro arbitrariamente pequeno, qualquer que seja o esquema de codificação.

C/T_c é chamado de **taxa crítica** — o limite fundamental de transmissão confiável. É importante notar que este é um **teorema de existência não-construtivo**: Shannon prova (via um argumento probabilístico de “codificação aleatória”) que um bom código existe, mas não fornece o algoritmo para construí-lo — encontrar códigos práticos que se aproximem desse limite foi um desafio de décadas de pesquisa em engenharia (culminando, décadas depois, em códigos como Turbo Codes e LDPC).

Caso especial — Canal Binário Simétrico (BSC): definindo a taxa de codificação $r = T_c/T_s$, a condição de confiabilidade se reduz a $r \leq C$.

10.3 Capacidade de canal: definição geral

$$C = \max_{p_X(x)} I(X, Y)$$

A capacidade é o **máximo da informação mútua** entre entrada e saída, otimizado sobre todas as distribuições de entrada possíveis. Depende **apenas** das probabilidades condicionais $P(y|x)$ que caracterizam o canal (não da entrada escolhida em si — é uma propriedade do canal).

Propriedades: $C \geq 0$; $C \leq \log |X|$ e $C \leq \log |Y|$; $I(X, Y)$ é contínua e **côncava** em $p_X(x)$ — o que garante que o máximo existe e é bem definido (problema de otimização côncava, sem múltiplos ótimos locais).

Canal Binário Simétrico (BSC)

Com probabilidade de erro (inversão de bit) p :

$$C = 1 - H(p)$$

onde $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ é a entropia binária. A curva $C(p)$ é simétrica em torno de $p = 0,5$: quando $p = 0$ (sem ruído), $C = 1$ bit/uso (máximo); quando $p = 0,5$, $C = 0$ — o canal é “inútil” (a saída não carrega nenhuma informação sobre a entrada, pois é indistinguível de ruído puro).

Canal com Apagamento (BEC) e canal combinado

Em um **canal binário com apagamento** (BEC), o receptor pode receber 0, 1, ou um símbolo de “apagamento” — o bit foi perdido, mas o receptor **sabe** que perdeu, sem confundir com outro valor. Para um canal que combina apagamento (probabilidade ϵ) com erro de inversão (probabilidade p , condicional a não ter sido apagado):

$$C = (1 - \epsilon)(1 - H(p))$$

Interpretação: $(1 - \epsilon)$ é a fração de bits que chegam (não apagados); $(1 - H(p))$ é a capacidade residual tipo BSC sobre os bits que chegaram. Casos particulares: $\epsilon = 0$ recai no BSC puro; $p = 0$ recai no BEC puro ($C = 1 - \epsilon$).

10.4 Terceiro Teorema de Shannon: capacidade do canal gaussiano (Shannon-Hartley)

Este é o resultado mais famoso de toda a Teoria da Informação. Considere um canal contínuo de largura de banda B Hz, com ruído aditivo gaussiano branco (AWGN) de densidade espectral $N_0/2$, e potência média de transmissão P .

Derivação (ideia central, passo a passo):

1. A saída é $Y_k = X_k + N_k$, com X_k e N_k independentes. A informação mútua é $I(X_k, Y_k) = H(Y_k) - H(Y_k|X_k) = H(Y_k) - H(N_k)$.
2. Como $H(N_k)$ não depende de como escolhemos codificar X_k , **maximizar I equivale a maximizar $H(Y_k)$** .
3. Para uma variância fixa, a entropia diferencial é máxima quando a distribuição é **gaussiana** (resultado do Capítulo 8) — logo, a entrada ótima X_k também deve ser gaussiana.
4. Com $Y_k \sim N(0, P + \sigma^2)$ e $N_k \sim N(0, \sigma^2)$ (onde $\sigma^2 = N_0 B$ é a potência do ruído), usando a fórmula de entropia diferencial gaussiana do Capítulo 8:

$$C = \frac{1}{2} \log_2 \left(1 + \frac{P}{\sigma^2} \right) \text{ bits por transmissão}$$

5. Como o canal é amostrado $2B$ vezes por segundo (teorema de Nyquist), a capacidade **por unidade de tempo** é:

$$C = B \log_2 \left(1 + \frac{P}{N_0 B} \right) \text{ bps}$$

Reescrevendo com a **razão sinal-ruído** $SNR = P/(N_0 B)$:

$$C = B \log_2(1 + SNR)$$

Leitura da fórmula

- C cresce apenas **logaritmicamente** com o SNR — dobrar a potência de transmissão não dobra a capacidade.
- C cresce **linearmente** com a largura de banda B (mas, atenção: aumentar B também aumenta o ruído total $N_0 B$ recebido, então o ganho não é ilimitado).
- É um **limite superior absoluto**: nenhum esquema de codificação, por mais sofisticado, consegue transmitir de forma confiável a uma taxa acima de C . Para se aproximar desse limite, o sinal transmitido deve ter estatísticas próximas às de ruído gaussiano branco — ou seja, o “código ótimo” tende a parecer ruído (dispersão máxima de energia, sem padrões redundantes exploráveis).

Exemplo numérico resolvido

Canal com $B = 5$ MHz e $SNR = 100$:

$$C = 5.000.000 \times \log_2(101) \approx 5.000.000 \times 6,658 \approx 33,29 \text{ Mbps}$$

Eficiência espectral e o limite de Shannon em E_b/N_0

Definindo E_b = energia por bit (com $P = E_b C$ em um “sistema ideal” onde a taxa de transmissão R_b iguala C) e a **eficiência espectral** $\eta = C/B$ (bits/s por Hz), pode-se reescrever a fórmula em termos de E_b/N_0 :

$$\frac{E_b}{N_0} = \frac{2^\eta - 1}{\eta}$$

Quando $\eta \rightarrow 0$ (banda muito larga em relação à taxa), esse limite converge para o famoso valor assintótico de $E_b/N_0 \approx -1,6$ dB — abaixo do qual **nenhuma** comunicação confiável é possível, não

importa quão sofisticada seja a codificação. Tecnologias modernas (4G/5G) usam técnicas como OFDM e MIMO precisamente para se aproximar desse limite teórico.

Capítulo 11 — Códigos Corretores de Erro

11.1 Motivação

Mesmo com um canal de alta capacidade, erros são inevitáveis (atenuação, interferência eletromagnética, imperfeições de hardware). A ideia central dos códigos corretores é **sacrificar eficiência de transmissão em troca de confiabilidade**, inserindo redundância controlada que permite ao receptor detectar e, em alguns casos, corrigir erros.

11.2 Categorias de códigos

- **Códigos de Hamming**: um dos exemplos mais antigos (Richard Hamming, anos 1950), projetados para corrigir automaticamente erros de bit único, adicionando bits de verificação que satisfazem relações matemáticas específicas.
- **Códigos Cíclicos**: a sequência codificada permanece válida mesmo após deslocamentos cíclicos — úteis para fluxos contínuos (TV digital, armazenamento).
- **Códigos Convolucionais**: geram redundância envolvendo a sequência de dados com sequências geradoras fixas — adequados para comunicação sem fio e espaço profundo.
- **Turbo Codes e LDPC** (*Low-Density Parity-Check*): códigos modernos de alto desempenho, usados em comunicação por satélite e redes sem fio avançadas, que se aproximam do limite de Shannon.

11.3 Técnicas de detecção e correção

Técnica	O que faz	Detecta?	Corrige?
Paridade	1 bit extra torna o total par/ímpar	Erros ímpares apenas	Não
Checksum	Soma das palavras de dados	Mais tipos de erro que paridade	Não
CRC	Trata os dados como polinômio, divide por polinômio gerador	Padrões de erro complexos	Não (por si só)

Técnica	O que faz	Detecta?	Corrige?
FEC (<i>Forward Error Correction</i>)	Redundância suficiente para correção sem retransmissão	Sim	Sim

11.4 Taxa de código e o trade-off fundamental

$$R = \frac{k}{n}$$

(bits de dados sobre bits totais após codificação). Taxa mais alta = menos redundância = maior eficiência de transmissão, mas **menor** capacidade de correção. É o mesmo trade-off central visto no Capítulo 10 ($r \leq C$): quanto mais se protege contra erro, menos “espaço útil” sobra no canal.

Exemplos de aplicação do trade-off: comunicação no espaço profundo (atraso enorme, retransmissão inviável) usa taxa de código **baixa** com forte correção; streaming de vídeo ao vivo (tolerância a pequenas perdas, alta demanda de tempo real) usa taxa **alta** com correção mais leve.

11.5 Aplicações modernas

- **Comunicação por satélite:** Turbo Codes e LDPC.
- **TV digital:** Códigos Cíclicos.
- **Armazenamento (HD/SSD):** Códigos de Hamming e Reed-Solomon.
- **Wi-Fi:** Códigos Convolucionais e Turbo Codes.

Capítulo 12 — Complexidade de Kolmogorov

12.1 Uma segunda forma de medir informação

Shannon mede informação através de **probabilidade**: a informação de um evento x é $-\log[p_X(x)]$. Em 1965, o matemático russo **Andrei Kolmogorov** propôs uma definição alternativa, sem qualquer referência a probabilidades: a **complexidade algorítmica** (ou descritiva) de um objeto é o comprimento do **menor programa** capaz de gerá-lo.

“A Complexidade de Kolmogorov mede a quantidade de informação de um objeto pelo tamanho do menor programa (algoritmo) capaz de produzi-lo.”

Exemplo canônico: compare duas sequências de 20 bits: - 11111111111111111111 — pode ser gerada por um programa curto: `FOR I=1 TO 20 PRINT 1`. **Baixa complexidade** (string “regular”/compressível). - 10110011010111011010 — não tem padrão aparente; o programa mais curto conhecido é essencialmente `PRINT 10110011010111011010`, quase do tamanho da própria string. **Alta complexidade** (string “aleatória”/incompressível).

Essa ideia conecta-se com a **Navalha de Occam** (Guilherme de Ockham, 1287–1347): entre explicações igualmente válidas, prefere-se a mais simples — e “mais simples” pode ser formalizado precisamente como “de menor complexidade de Kolmogorov”.

12.2 Definição formal

Seja U um computador (interpretador) universal, p um programa e $U(p)$ a saída de U ao executar p . A **Complexidade de Kolmogorov** de uma string x é:

$$K_U(x) = \min_{p: U(p)=x} l(p)$$

isto é, o comprimento do menor programa que produz x como saída.

Teorema da Invariância (existência de função ótima)

A definição parece depender da escolha do interpretador U — mas essa dependência é **limitada**: existe uma função U (“ótima” ou “universal”) tal que, para qualquer outro interpretador A , existe uma constante c_A (independente de x !) tal que

$$K_U(x) \leq K_A(x) + c_A$$

Prova (ideia): dada a descrição mais curta t de x relativa a A (ou seja, $A(t) = x$), o interpretador universal U pode simular A : basta um programa (p, t) onde p é um “interpretador de A ” escrito na linguagem de U (tamanho fixo $|p|$) seguido da descrição t . Logo $K_U(x) \leq |p| + |t| = c_A + K_A(x)$. ■

Consequência: **todas as funções ótimas dão essencialmente a mesma complexidade**, a menos de uma constante aditiva — por isso a notação $K(x)$ (sem subscrito) é usada por convenção, entendendo-se que há uma constante de ambiguidade fixa, independente de x .

Um limite superior trivial: $K(x) \leq |x| + C$ — basta usar a própria string x como sua descrição (o programa “imprima literalmente isto”).

12.3 Strings incompressíveis

Proposição: a fração de strings de comprimento n com $K(x) < n - k$ não excede 2^{-k} .

Prova: há 2^n strings de comprimento n , mas apenas menos de 2^{n-k} descrições potenciais de comprimento $< n - k$. ■

Consequência direta: **existem strings incompressíveis** (com $K(x) \geq n$) — há 2^n strings de comprimento n , mas no máximo $2^n - 1$ descrições de comprimento $< n$, logo pelo menos uma string não admite descrição mais curta que si mesma.

Curiosidade paradoxal: não é possível **encontrar efetivamente** (por algoritmo) uma string incompressível de comprimento n dado. Se fosse possível, o próprio algoritmo (mais o valor de n , que precisa de $\approx \log n$ bits) seria uma descrição de tamanho $\log n + c$ — e para n grande, $\log n + c < n$, contradizendo a incompressibilidade. Esse mesmo argumento de contagem mostra por que **nenhum algoritmo de compressão pode comprimir todo arquivo possível**: não há espaço suficiente de descrições curtas para todas as entradas.

12.4 O Teorema de Gödel via Complexidade de Kolmogorov

Uma das aplicações mais espetaculares desta teoria: uma prova do **Teorema da Incompletude de Gödel** em poucas linhas (formulação sugerida por Gregory Chaitin).

Teorema: existe um número m tal que, para todo x , a proposição “ $K(x) \geq m$ ” **não é demonstrável** — mesmo sendo verdadeira para infinitos valores de x (já que só há finitas descrições de comprimento $< m$).

Prova (por absurdo): suponha que, para todo m , exista algum x tal que “ $K(x) \geq m$ ” seja demonstrável. Um algoritmo hipotético, dado m , poderia enumerar todos os teoremas de um sistema formal até encontrar um da forma “ $K(x) \geq m$ ” e retornar x . Isso faria de m uma **descrição** de x (via esse algoritmo fixo), logo $K(x) \leq \log m + C$. Mas por hipótese “ $K(x) \geq m$ ” é verdadeiro (todo teorema demonstrado é verdadeiro) — logo $m \leq \log m + C$, desigualdade **falsa para m grande**. Contradição. ■

Esse argumento formaliza o **paradoxo de Berry** (Bertrand Russell, 1908): “*o menor inteiro que precisa de mais de mil palavras para ser descrito*” — uma frase que parece descrever esse

número usando poucas palavras, gerando um paradoxo aparente. A resolução está em distinguir “menor elemento com a propriedade P ” (bem definido) de “descrição” (que só a Complexidade de Kolmogorov formaliza rigorosamente): a versão corrigível é “o primeiro inteiro tal que ‘ $K(N) > m$ ’ é demonstrável” — o mesmo argumento do Teorema de Gödel.

12.5 Aleatoriedade formalizada sem probabilidade

Pergunta motivadora: por que uma placa de carro “7777 ZZ 77” parece mais “extraordinária” que “7353 NY 42”, se ambas têm exatamente a mesma probabilidade sob um modelo uniforme? A resposta intuitiva: a primeira pertence a um **conjunto pequeno e facilmente descritível** (“dígitos repetidos”), enquanto a segunda não tem nenhuma propriedade especial que a coloque em um conjunto pequeno — ela é “apenas um número”. Essa intuição é exatamente o que a Complexidade de Kolmogorov formaliza.

Definição (sequências finitas): uma string é “aleatória” na medida em que sua complexidade se aproxima do seu próprio comprimento — a melhor forma de descrevê-la é simplesmente escrevê-la por extenso.

Definição (sequências infinitas): requer cuidado técnico adicional (a noção de **complexidade prefixa**, que evita a necessidade de codificar separadamente o comprimento da descrição). Com essa correção, x é dita aleatória se $\exists C \forall n : K(x_{1:n}) > n - C$.

Equivalência de Levin-Schnorr (anos 1970): a **incompressibilidade** de uma sequência infinita e sua **resistência a todo teste algorítmico de aleatoriedade** (no sentido de Martin-Löf) são **equivalentes** — um dos resultados centrais da Teoria Algorítmica da Informação, que permite reformular teoremas clássicos de probabilidade (“para quase todo x , vale P ”) como afirmações sobre sequências aleatórias individuais (“para toda sequência aleatória, vale P ”).

Curiosidade: uma sequência infinita x é **computável** (recursiva) se e somente se $\exists C \forall n : K(x_{1:n} | n) < C$ (Teorema de Meyer) — as sequências mais “simples” são exatamente as computáveis.

12.6 Complexidade de Kolmogorov como precursor da entropia

Fechando a conexão com o restante do curso: para uma variável aleatória X , Shannon define a informação de um evento $X = x$ como $-\log[p_X(x)]$; a **complexidade descritiva** correspondente é $\lceil -\log[p_X(x)] \rceil$ — o comprimento necessário para codificar x usando um código de Shannon ótimo. A Complexidade de Kolmogorov generaliza essa ideia **sem depender de uma distribuição de probabilidade** conhecida a priori: mede a informação pelo tamanho do menor programa, não pela probabilidade do evento. Notavelmente, para a maioria dos objetos, o valor esperado da Complexidade de Kolmogorov **coincide aproximadamente** com a entropia de Shannon — por isso ela é considerada um **fundamento alternativo** (mais geral, e não-probabilístico) do próprio conceito de informação.

PARTE III — BIG DATA, APRENDIZADO DE MÁQUINA E RECONHECIMENTO DE PADRÕES

Capítulo 13 — Big Data: Frameworks e Processamento Distribuído

13.1 Por que processamento distribuído?

Quando o volume de dados excede a capacidade de uma única máquina, é preciso **distribuir** o processamento entre vários nós de um cluster. O framework clássico para isso é o **MapReduce**.

13.2 MapReduce

Ideia central: o programador define apenas duas funções — **Map** e **Reduce** — e o framework cuida de toda a complexidade de paralelização, particionamento e comunicação entre nós. Os dados fluem como tuplas (**chave**, **valor**).

$$\text{map} : (k_1, v_1) \rightarrow \text{list}(k_2, v_2), \quad \text{reduce} : (k_2, \text{list}(v_2)) \rightarrow \text{list}(k_3, v_3)$$

Desenvolvido pelo Google (Dean & Ghemawat, 2004); a implementação *open-source* mais popular é o **Apache Hadoop**.

Exemplo canônico: WordCount

Contar a ocorrência de cada palavra em um conjunto de textos:

```
map(line_number, text):
    for each word in word_list:
        emit(word, 1)

reduce(word, values[]):
    word_count = sum(values)
    emit(word, word_count)
```

O **Map** emite (**palavra**, 1) para cada ocorrência; o **Reduce** soma todos os valores associados à mesma chave — essa é, em essência, a operação de **Agregação** da álgebra relacional.

Operações da álgebra relacional via MapReduce

Operação	Map	Reduce
Seleção (σ_C)	Testa condição C ; emite (t, t) se verdadeira	Identidade
Projeção (π_S)	Remove atributos fora de S ; emite (t', t')	Remove duplicatas
União	Emite (t, t) de ambas as relações	Remove duplicatas
Interseção	Emite (t, t) de ambas	Emite só se presente nas duas
Diferença ($P - S$)	Marca origem (t, P) ou (t, S)	Emite se só está em P
Junção natural ($P \bowtie S$)	Emite $(b, P(a))$ e $(b, S(c))$ pela chave comum b	Combina pares (a, c) por chave
Agrupamento/Agregação	Emite $(chave, valor)$	Agrega (soma etc.) por chave

Custo computacional

O custo de comunicação costuma **dominar** o custo de processamento em si (ler do disco para memória é mais caro que a computação). Para um *join* $R(A, B) \bowtie S(B, C)$ com tamanhos r, s , o custo de comunicação é $O(r + s)$.

Exemplo — multiplicação de matrizes $n \times n$: com taxa de replicação $r = n$ e tamanho de redutor $q = 2n$, obtém-se $qr \geq 2n^2$, ou seja, limite inferior $\Omega(n^2)$ — melhor que o algoritmo recursivo clássico $\Theta(n^3)$.

13.3 Processamento em fluxo (Streaming) e Apache Spark

Enquanto o MapReduce clássico processa dados em **lote** (*batch*), o **Streaming Processing** é orientado a eventos: fluxo contínuo de pequenos pacotes, alto número de transações por segundo, em modo de transmissão imprevisível (“*burst mode*”), com execuções **quase em tempo real** (critério típico: 90%+ das operações concluídas em menos de 1 segundo).

O **Apache Spark** processa dados **inteiramente em memória** (ao contrário do Hadoop clássico, que grava resultados intermediários em disco), usando a abstração de **RDD** (*Resilient Distributed Dataset*), com suporte a streaming via **DStream**. Um **SparkContext** (dentro do *driver program*) coordena o cluster via um *Cluster Manager*, que distribui o trabalho entre *workers* executando *executores*.

13.4 Convergência com IA

Big Data e Inteligência Artificial são vistas como **técnicas convergentes**: IA se beneficia de plataformas de Big Data (PaaS, GPU clusters) para viabilizar treinamento em larga escala (pipelines de **ETL** — *Extract, Transform, Load*). Um exemplo prático: acelerar o Spark com GPUs via bibliotecas RAPIDS (NVIDIA).

Edge Computing e Federated Learning: em cenários de IoT, centralizar todo o dado para treinar um modelo gera latência e problemas de privacidade. O **Aprendizado Federado** resolve isso treinando modelos localmente em cada dispositivo de borda; apenas as **atualizações de parâmetros** (não os dados brutos) são enviadas a um servidor central, que agrega os modelos (ex.: algoritmo FedAvg) — reduzindo tráfego de rede e preservando privacidade.

Capítulo 14 — Fundamentos de Redes Neurais e Aprendizado

14.1 Por que aprendizado de máquina?

Sistemas especialistas (baseados em regras IF-THEN explícitas) funcionam bem quando as regras são claras, mas falham em tarefas perceptualmente simples para humanos, porém difíceis de codificar explicitamente (ex.: reconhecer o que há em uma imagem). A alternativa é imitar como humanos aprendem: **exposição a muitos dados + exemplos rotulados (labels) → o próprio sistema identifica padrões.**

Abordagem	Como funciona	Exige conhecimento de domínio?
Sistemas especialistas	Regras explícitas (IFs)	Sim, muito detalhado
Redes Neurais	Identifica <i>features</i> automaticamente	Exige dados bem estruturados
Deep Learning	Múltiplas camadas de <i>features</i> (hierárquicas)	Não exige estrutura fina, mas exige muito dado

Features (atributos de entrada): pixels (classificação de imagem), tom/volume (reconhecimento de voz), dados de câmeras/GPS (carros autônomos). Extrair *features* relevantes é essencial: uma feature irrelevante (ex.: hora do dia para classificar imagens de gatos) não ajuda; uma feature relevante em outro contexto (hora do dia para detectar SPAM, já que e-mails de SPAM costumam ser enviados de madrugada) pode ser crucial.

14.2 Métricas de avaliação

$$\text{Precisão} = \frac{VP}{VP + FP}, \quad \text{Recall} = \frac{VP}{VP + FN}, \quad \text{Acurácia} = \frac{VP + VN}{\text{total}}$$

(VP = verdadeiro positivo, FP = falso positivo, FN = falso negativo, VN = verdadeiro negativo)

14.3 O mecanismo de treinamento

Erro e função de perda

$$MSE = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2, \quad RMSE = \sqrt{MSE}$$

O MSE é o segundo momento do erro (combina viés e variância do estimador) e é sensível a *outliers*. Treinar um modelo é, essencialmente, **minimizar uma função de perda** (*loss*) — no caso mais simples, o próprio MSE.

Gradiente descendente

$$w \leftarrow w - \alpha \cdot \frac{\partial L(w)}{\partial w}$$

onde α é a **taxa de aprendizagem** (*learning rate*). Um α muito alto causa divergência/oscilação; um α muito baixo torna a convergência lenta. Vocabulário associado: **época** (uma passada completa pelo dataset), **batch** (subconjunto de amostras usado por atualização), **step** (uma atualização de pesos).

Variantes do Gradiente Descendente

Variante	Fórmula	Características
Vanilla (Batch) GD	$\theta = \theta - \eta \nabla_{\theta} J(\theta)$ (dataset inteiro)	Lento, garante convergência ao mínimo global (em funções convexas)
SGD (estocástico)	$\theta = \theta - \eta \nabla_{\theta} J(\theta; x^{(i)}, y^{(i)})$ (1 exemplo)	Rápido, mas oscila em torno do mínimo
Mini-batch GD	Sobre um lote de n amostras (tipicamente 32–64)	Equilíbrio entre velocidade e estabilidade — o mais usado na prática

Para acelerar a convergência em superfícies de perda “estreitas” (ravinas), usa-se **Momentum**: $v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$, que acumula uma fração γ do gradiente anterior. Variantes mais avançadas (Adam, Adagrad, RMSprop) adaptam a taxa de aprendizagem por parâmetro.

Funções de ativação (a não-linearidade)

Sem uma função de ativação não-linear, uma rede de múltiplas camadas equivaleria a uma **única** transformação linear (composição de funções lineares é linear) — é a não-linearidade que dá às redes neurais seu poder expressivo.

$$\text{ReLU}(x) = \max(0, x), \quad \text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}, \quad \text{Softmax}(z_j) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

Backpropagation (regra da cadeia)

Uma rede é uma composição de módulos $X_i = F_i(X_{i-1}, W_i)$. Treinar significa calcular $\partial E / \partial W_i$ para **todas** as camadas, usando a regra da cadeia:

$$\frac{\partial E}{\partial W_i} = \frac{\partial E}{\partial X_i} \cdot \frac{\partial F_i(X_{i-1}, W_i)}{\partial W_i}$$

O algoritmo de **retropropagação** (*backpropagation*) calcula isso eficientemente varrendo a rede da saída para a entrada: define-se $\delta_n = \partial C / \partial X_n$ na última camada, e propaga-se recursivamente $\delta_{i-1} = \delta_i \cdot \partial F_i / \partial X_{i-1}$, obtendo $\partial E / \partial W_i = \delta_i \cdot \partial F_i / \partial W_i$ em cada camada.

14.4 Overfitting e boas práticas

Overfitting: quando um modelo se ajusta excessivamente aos dados de treino, capturando ruído em vez de padrão real, e por isso **não generaliza** bem para dados novos — detectável comparando o erro (RMSE) em treino vs. validação.

Conjuntos de dados: | Conjunto | Papel | Proporção típica | |—|—|—| | Treinamento | Ajusta os pesos | ~80% | | Validação | Avalia sem viés durante o ajuste (cross-validation) | ~20% | | Teste | Avaliação final, usada uma única vez | Conjunto separado e curado |

Batch Normalization: normalizar (Capítulo 3) apenas uma vez, antes do treino, “se perde” ao longo das atualizações de peso. Normalizar **dentro de cada batch**, durante o treino, resolve esse problema e costuma reduzir o número de épocas necessárias para convergir.

Capítulo 15 — Análise de Componentes Principais (PCA)

15.1 Contexto histórico

O **PCA** (*Principal Component Analysis*) foi desenvolvido por **Harold Hotelling** em **1933** (também chamado de **Transformada de Hotelling**); em processamento de sinais é conhecido como **Transformada de Karhunen-Loève**. É um caso particular da **Decomposição em Valores Singulares (SVD)** quando a matriz decomposta é a matriz de covariância dos dados — que é sempre quadrada, simétrica e definida positiva.

15.2 O problema: redução de dimensionalidade preservando informação

Dado um conjunto de N vetores de atributos $x_k \in \mathbb{R}^p$ organizados em uma matriz $X \in \mathbb{R}^{p \times N}$, o objetivo é encontrar uma transformação linear $z_k = Qx_k$ que produza vetores $z_k \in \mathbb{R}^q$ com $q \leq p$, preservando o máximo de informação relevante contida em X .

15.3 O algoritmo PCA, passo a passo

Passo 1 — Vetor-média: $\bar{x} = \frac{1}{N} \sum_{k=1}^N x_k$

Passo 2 — Centralização: $x_k \leftarrow x_k - \bar{x}$ (daqui em diante, x_k já está centralizado)

Passo 3 — Matriz de covariância: $C_x = E[xx^T] \approx \frac{1}{N} \sum_{k=1}^N x_k x_k^T$

Passo 4 — Autovalores e autovetores: resolver o problema de autovalor $C_x v = \lambda v$, cujas soluções não-triviais exigem

$$\det(C_x - \lambda I_p) = 0$$

Como C_x é simétrica e definida positiva, **todos os p autovalores são reais e positivos**. Convenciona-se ordená-los de forma decrescente: $\lambda_1 > \lambda_2 > \dots > \lambda_p$.

Exemplo numérico resolvido ($p = 2$):

$$C_x = \begin{bmatrix} 1 & 0,8 \\ 0,8 & 4 \end{bmatrix} \Rightarrow \det(C_x - \lambda I) = \lambda^2 - 5\lambda + 3,36 = 0 \Rightarrow \lambda_1 = 4,2, \lambda_2 = 0,8$$

Nota de correção: alguns slides do curso invertem a ordem ($\lambda_1 = 0,8$, $\lambda_2 = 4,2$) — mas por **convenção**, λ_1 é sempre o **maior** autovalor.

Os autovetores v_i formam um conjunto **ortonormal** ($v_i^T v_j = 1$ se $i = j$, 0 caso contrário), dispostos em colunas na matriz $V = [v_1 \ v_2 \ \dots \ v_p]$. Como $VV^T = I_p$, tem-se $V^{-1} = V^T$ — propriedade essencial para reconstruir os dados originais a partir dos transformados.

Observação sobre não-unicidade: se v_i é solução, $-v_i$ também é (basta verificar: $C_x(-v_i) = \lambda_i(-v_i)$) — diferentes implementações numéricas (ex.: `eig` vs. `pcacov` no Octave/MATLAB) podem retornar autovetores com sinais diferentes, sem que isso afete a análise.

Passo 5 — Matriz de transformação: $Q = V^T$

Passo 6 — Transformação: $z_k = Qx_k$ (ou, em forma matricial, $Z = QX$). O mapeamento inverso é $x_k = Q^T z_k$ (ou $X = Q^T Z$).

15.4 Por que isso funciona? A diagonalização da covariância

O resultado central do PCA é entender o que acontece à covariância dos dados **após** a transformação:

$$C_z = E[zz^T] = E[(V^T x)(V^T x)^T] = V^T E[xx^T]V = V^T C_x V$$

Usando $C_x v_i = \lambda_i v_i$ e a ortonormalidade dos v_i :

$$C_z = V^T C_x V = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_p)$$

Conclusões-chave: 1. A covariância dos dados transformados é **diagonal** — as componentes de z são **descorrelacionadas** entre si. 2. As **variâncias** das novas variáveis z_i são exatamente os **autovalores** de C_x : $\lambda_i = \sigma_i^2$. Não é preciso recalcular nada — os autovalores já foram obtidos no Passo 4.

15.5 Interpretação geométrica: mudança de base

Uma **base** de $\mathbb{R}^p$ é qualquer conjunto de p vetores linearmente independentes; uma base é **ortonormal** se seus vetores são mutuamente ortogonais e de norma unitária. Os dados originais x_k estão representados na **base canônica**; os dados transformados z_k estão representados na base formada pelos **autovetores da matriz de covariância**.

Geometricamente, o PCA corresponde a uma **rotação do sistema de coordenadas**: os novos eixos ($s_1, s_2, \dots$) se alinham com as direções de maior variância dos dados. No espaço original, os atributos costumam estar correlacionados (nuvem de pontos “inclinada”); depois do PCA, a nuvem fica alinhada aos eixos (descorrelacionada) — o primeiro eixo captura a direção de máxima dispersão, o segundo a maior dispersão *ortogonal* ao primeiro, e assim sucessivamente.

15.6 Aplicações

Seleção de atributos

A primeira componente $z_1 = v_1^T x = v_{11}x_1 + v_{21}x_2 + \dots + v_{p1}x_p$ é a projeção dos dados na direção de maior variância. Quanto maior o coeficiente $|v_{j1}|$, **mais importante** é a variável original x_j para explicar a variância principal dos dados — usado como critério de seleção de atributos.

Redução de dimensionalidade

Usando apenas as q primeiras colunas de V (associadas aos q maiores autovalores), forma-se V_q ($p \times q$) e $Q_q = V_q^T$. A transformação $z_k = Q_q x_k$ produz vetores de dimensão $q < p$ — uma **compressão com perdas**, análoga em espírito à compressão de dados do Capítulo 9, mas para dados numéricos multivariados em vez de símbolos discretos.

Variância total e variância explicada

$$VT = \sum_{i=1}^p \lambda_i \quad (\text{variância total, medida linear de "informação"})$$

$$VE_i = 100 \times \frac{\lambda_i}{VT} \quad (\text{variância explicada pela componente } i), \quad VE(q) = 100 \times \frac{\sum_{i=1}^q \lambda_i}{VT} \quad (\text{acumulada até } q)$$

O gráfico de $VE(q)$ em função de q é o **scree plot**, usado para decidir quantas componentes reter (tipicamente onde a curva “dobra”). Se $q = p$, não há perda de informação (reconstrução perfeita); se $q < p$, há perda proporcional à variância descartada — um trade-off explícito entre compressão e fidelidade, no mesmo espírito da compressão com perdas do Capítulo 9.

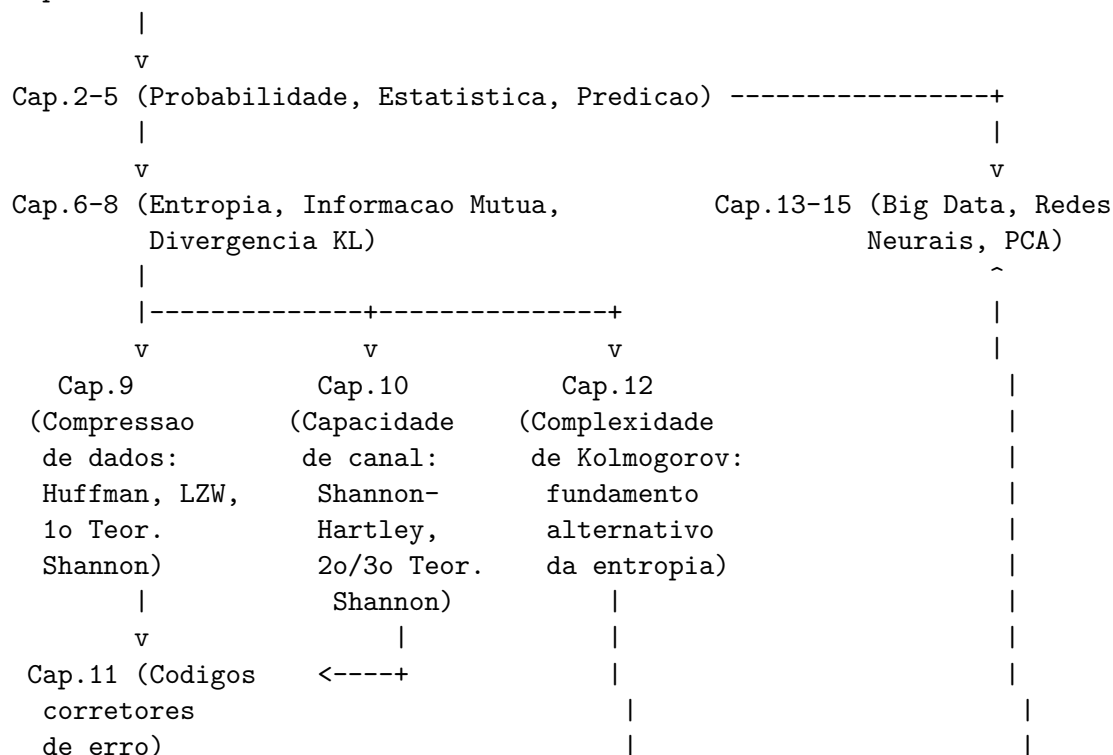
Capítulo 16 — Síntese Final e Mapa Conceitual

16.1 O fio condutor do curso

Se há uma ideia que amarra todos os quinze capítulos anteriores, é esta: **informação pode ser quantificada matematicamente, e essa quantificação tem consequências práticas precisas** — quantos bits, em média, um símbolo carrega (entropia); quantos bits, no mínimo, uma mensagem pode ser comprimida (Primeiro Teorema de Shannon); quantos bits por segundo um canal ruidoso consegue transportar de forma confiável (Shannon-Hartley); quantos bits, no mínimo, descrevem um objeto (Complexidade de Kolmogorov); e quantas dimensões, no mínimo, capturam a maior parte da variância de um conjunto de dados (PCA).

16.2 Mapa de conexões entre os capítulos

Cap.1 (Dados -> Informacao -> Conhecimento)



(ganho de informacao, KLD como
funcao de perda em ML, PCA como
"compressao" de dados numericos)

16.3 Tabela-resumo das grandezas fundamentais

Grandeza	Fórmula essencial	O que responde
Autoinformação $I(a_k)$	$-\log_2 p_k$	Quanta surpresa carrega <i>um</i> evento?
Entropia $H(A)$	$-\sum_k p_k \log_2 p_k$	Quantos bits, em média, por símbolo?
Entropia condicional $H(A B)$	$H(A, B) - H(B)$	Quanta incerteza sobre <i>sobre</i> A , sabendo B ?
Informação mútua $I(A, B)$	$H(A) - H(A B)$	Quanto B me diz sobre A ?
Divergência KL $D(p\ g)$	$\sum p \log(p/g)$	Quão diferente é p de g ?
Comprimento médio ótimo L_{min}	$= H(A)$	Menor taxa média de compressão sem perdas
Capacidade de canal C	$B \log_2(1 + SNR)$	Máxima taxa confiável de transmissão
Complexidade de Kolmogorov $K(x)$	$\min\{l(p) : U(p) = x\}$	Menor programa que gera x
Variância explicada $VE(q)$	$\sum_{i \leq q} \lambda_i / \sum_i \lambda_i$	Quanta informação (linear) as q primeiras componentes retêm?

16.4 Dicas de estudo e revisão

- **Para quem está vendo pela primeira vez:** siga a ordem dos capítulos. A Parte I (probabilidade e estatística) é pré-requisito indispensável para a Parte II; a Parte II, por sua vez, fundamenta boa parte do vocabulário usado na Parte III (ganho de informação em árvores de decisão, divergência KL como função de perda, entropia como critério de otimização).
- **Para quem está revisando:** a Tabela-resumo (16.3) e o Mapa de Conexões (16.2) servem como índice rápido; cada capítulo tem exemplos numéricos resolvidos que podem ser refeitos como exercício de fixação (Huffman no Cap. 9, Shannon-Hartley no Cap. 10, PCA 2×2 no Cap. 15, cadeia de Markov no Cap. 5).
- **Erros e imprecisões dos materiais originais:** ao longo da consolidação deste texto, alguns pontos dos slides originais foram corrigidos ou esclarecidos (ex.: direção da desigualdade de Jensen, ordenação de autovalores no exemplo de PCA, a equação 58 de entropia condicional diferencial). Essas correções estão sinalizadas em caixas de nota ao longo do texto.

Referências e leituras complementares

Materiais originais da disciplina (Prof. Dr. Julio César Santos dos Anjos): - Módulos 01–06 de Tecnologia da Informação (slides). - Métricas — Revisão — Pré-Processamento de Dados. - Apostila UFC — Introdução à Teoria da Informação e Funções Estatísticas Importantes. - Análises de Componentes Principais — PCA — Principais Conceitos. - Complexidade de Kolmogorov (capítulo de Bruno Durand e Alexander Zvonkin).

Bibliografia citada nos materiais e recomendada para aprofundamento: - SHANNON, C. E. *A Mathematical Theory of Communication*. Bell System Technical Journal, 1948. - COVER, T. M.; THOMAS, J. A. *Elements of Information Theory*. Wiley (referência clássica para aprofundar entropia diferencial, capacidade de canal e os teoremas de Shannon em forma matemática precisa). - BISHOP, C. M. *Pattern Recognition and Machine Learning*. Springer, 2009. - HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. *The Elements of Statistical Learning*. 2011. - ROSS, S. *Probabilidade: Um Curso Moderno com Aplicações*. 2010. - LESKOVEC, J.; RAJARAMAN, A.; ULLMAN, J. D. *Mining of Massive Datasets*. 3ª ed., Cambridge University Press, 2020. - DEAN, J.; GHEMAWAT, S. *MapReduce: Simplified Data Processing on Large Clusters*. OSDI, 2004. - KOLMOGOROV, A. N. *Three Approaches to the Definition of the Concept of “Quantity of Information”*. 1965. - HOFSTADTER, D. *Gödel, Escher, Bach: An Eternal Golden Braid*. 1979. - HOTELLING, H. *Analysis of a Complex of Statistical Variables into Principal Components*. Journal of Educational Psychology, 1933. - RÉNYI, A. *On Measures of Information and Entropy*. Proceedings of the 4th Berkeley Symposium, 1960.

Apostila consolidada a partir dos materiais de curso e organizada para uso como referência única de estudo e revisão da disciplina de Teoria da Informação.